A project report on

FACE RECOGNITION SYSTEM AND SMART IMAGE MANAGEMENT

Submitted in partial fulfillment for the award of the degree of

Master of Sciences
In
Data science

by

PATNALA SIVA SUBRAMANYAM

(21BSD7020)

VIT-AP UNIVERSITY

School of Advanced Sciences

MAY 2026

A project report on

FACE RECOGNITION SYSTEM AND SMART IMAGE MANAGEMENT

Submitted in partial fulfillment for the award of the degree of

Master of Sciences
In
Data science

by

PATNALA SIVA SUBRAMANYAM

(21BSD7020)

VIT-AP UNIVERSITY

School of Advanced Sciences

MAY 2026

DECLARATION

I hereby declare that the internship report entitled "Face Recognition System and Smart Image Management" submitted by me for the award of the degree of Integrated M.Sc Data Science at VIT-AP University, Amaravati, is a record of bona fide work carried out by me during the period of internship at Snapwelt (Arisis Technologies Private Limited), HITEC City, Hyderabad, under the supervision and guidance of Mr. Turupati Chandra Shekhar, Founder & CEO.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university

Place: Amaravati
Date: 5 May 2026

Patnala Siva Subramanyam
Signature of the Candidate

Reference Code: SW-0041
Subject: INTERNSHIP COMPLETION CERTIFICATE
Date: 20/05/2026

Mr. Patnala Siva Subramanyam,
Hyderabad.

Re: Internship Completion Certificate at **Snapwelt** (Arisis Technologies Private Limited).

This is to certify that **Mr. Patnala Siva Subramanyam** Final Year student, from Vellore Institute of Technology, Andhra Pradesh, Amaravati has completed internship as **Machine Learning Intern.**

The duration of internship was from **20 January 2026 to 20 May 2026**. During this period, he successfully completed the assigned tasks within the stipulated time.

The project was submitted to the company by the end of May 2026.

We wish you a bright and successful future, and look forward to a mutually fruitful association.

For Snapwelt (Arisis Technologies Private Limited)

Chandra Shekar Turupati,
Founder & CEO

ARISIS TECHNOLOGIES PRIVATE LIMITED
CIN : U72900TG2021PTC147675
PLOT NO. 931, 102, AYUSH AVENUE, RD NUMBER 48, OPP. CJR INTERNATIONAL SCHOOL, AYYAPPA SOCIETY, HITEC CITY, MADHAPUR, TELANGANA 500081

SUPPORT@SNAPWELT.COM |+91 9959655626 | WWW.SNAPWELT.COM

CERTIFICATE

This is to certify that the thesis work titled "**FACE RECOGNITION SYSTEM AND SMART IMAGE MANAGEMENT**" submitted by PATNALA SIVA SUBRAMANYAM, bearing registration number 21BSD7020 from the School of Advanced Science, VIT-AP University, is in partial fulfillment of the requirements for the award of the degree of Master of science in Data science, is a record of Bonafide work done under my guidance. The contents of this thesis have not been submitted and will not be submitted either in parts or in full, for the award of any other degree or diploma in this Institute or any other Institute or University.

External Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner

External Examiner

Approved by

Program Coordinator
Master of Science

Dean School of
Advanced Science

ABSTRACT

This internship report presents the comprehensive design, implementation, and deployment of a Face Recognition System and Smart Image Management platform, developed during a four-month internship at Snapwelt (Arisis Technologies Private Limited), Hyderabad. The project addresses the growing industrial demand for scalable, accurate, and intelligent facial biometric systems capable of operating across large-scale image databases in real time.

The system is architected around a multi-stage processing pipeline. Face detection is accomplished using the RetinaFace model integrated within the InsightFace framework, which delivers robust multi-scale face localization with sub-pixel landmark alignment even under challenging conditions such as partial occlusion, extreme lighting, and off-axis facial orientations. Detected face regions are subsequently forwarded to the ArcFace recognition model, which generates 512-dimensional L2-normalized facial embedding vectors that encode deep discriminative identity-level features. These embeddings are stored and indexed within a FAISS (Facebook AI Similarity Search) index using the IndexFlatIP structure, enabling exact inner product similarity computation with sub-50-millisecond retrieval latency across databases containing thousands of facial embeddings.

The smart image management module extends beyond one-to-one face verification by incorporating DBSCAN (Density-Based Spatial Clustering of Applications with Noise) clustering on the stored embedding space, enabling automatic grouping of images by detected identity without requiring prior knowledge of the number of individuals present. This unsupervised clustering approach is particularly suitable for real-world deployment scenarios where the total number of identities is dynamic and unknown.

The system achieves a face detection accuracy of 99.8% on benchmark test sets, with an average face search latency of approximately 50 milliseconds across a database of 10,000 indexed embeddings. The Gradio-based web interface provides an accessible

and intuitive interaction layer, enabling non-technical users to perform face search, registration, album organization, and identity clustering through a browser-based application without requiring any local installation.

This report details every stage of the system development lifecycle, encompassing requirement analysis, literature review, architectural design decisions, algorithmic implementation, performance evaluation, challenge resolution, and future development directions. The work represents a significant contribution to the practical application of deep learning-based facial biometrics for intelligent image organization and retrieval.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Mr. Turupati Chandra Shekhar, Founder & CEO of 'Snapwelt (Arisis Technologies Private Limited)' for their constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association With him is not confined to academics/internship only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Data Science.

I would like to express my gratitude to, Dr. G. VISWANATHAN CHANCELLOR, Dr. SEKHAR VISWANATHAN VICE PRESIDENT, Dr. ARULMOZHIVARMAN VICE CHANCELLOR, PROF.S SRINIVAS DEAN SCHOOL OF ADVANCED SCIENCES(SAS), for providing with an environment to work in and for his inspiration during the tenure of the course.

In jubilant mood I express ingeniously my wholehearted thanks to Dr. Faizan Danish, (Program Coordinator Dual Degree B.Sc. - MSc. Data Science) Assistant Professor, all teaching staff and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

It Is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Amaravati **Patnala Siva Subramanyam**

Date: 05-05-2026 Name of the student

TABLE OF CONTENTS

Declaration iii
Certificate iv
Abstract vi
Acknowledgement viii
Table of Contents ix
List of Figures xi
List of Tables xii
List of Abbreviations xiii
Chapter 1 – Introduction
1.1 Background and Context……...1
1.2 Problem Statement……2
1.3 Objectives of the Project……...3
1.4 Scope of the Work……4
1.5 Motivation for the Study……...5
Chapter 2 – Literature Review
2.1 Evolution of Face Recognition Technology…… 7
2.2 ArcFace: Additive Angular Margin Loss…….....8
2.3 InsightFace Framework……..10
2.4 RetinaFace and MTCNN for Face Detection……11
2.5 FAISS: Efficient Similarity Search……12
2.6 DBSCAN Clustering Algorithm…… 14
2.7 Limitations of Prior Work……..15
Chapter 3 – System Architecture and Design
3.1 Hardware and software Specifications……17
3.2 High-Level System Overview……17
3.3 System Architecture Diagram…… 18
3.4 Data Flow Diagram…… 19
3.5 Face Recognition Pipeline Flowchart……... 19
3.6 Module Descriptions…….. 21
Chapter 4 – Implementation
4.1 Development Environment…… 25
4.2 Face Detection Pipeline……..25
4.3 Embedding Generation with ArcFace……27
4.4 FAISS Indexing and Search……... 29
4.5 DBSCAN Clustering……..31
4.6 Gradio Web Interface……33

Chapter 5 – Results and Analysis
5.1 Performance Metrics…….. 35
5.2 Threshold Analysis…….38
5.3 Model Comparisons……... 40
5.4 Clustering Results…….. 41
Chapter 6 – Challenges and Solutions
6.1 FAISS GPU Compatibility…….44
6.2 Backend Fallback Strategy…….45
6.3 DBSCAN Parameter Tuning……..46
6.4 Face Quality and Edge Case Handling ……..47
Chapter 7 – Conclusion and Future Work
7.1 Summary of Findings…….49
7.2 Contributions of this study...……... 50
7.3 Future Directions……51
7.4 Source Code and Output…… 53
References ……..57
Appendices …….61

LIST OF FIGURES

Figure 3.3: System Architecture Diagram…… ……1
Figure 3.4: Data Flow Diagram – End-to-End Processing ……...19
Figure 3.5: Face Recognition Pipeline Flowchart…….20
Figure 4.2: RetinaFace Detection Output with Landmarks …….26
Figure 4.3: ArcFace Embedding Space Visualization (t-SNE)……28
Figure 4.4: FAISS IndexFlatIP Search Result Structure……..30
Figure 4.5: DBSCAN Clustering Output Visualization……...31
Figure 4.6.1: Gradio Web Interface – Main Dashboard……..33
Figure 4.6.2: Gradio Interface – Face Search Tab……...34
Figure 4.6.3: Gradio Interface – Album Clustering Tab……..34
Figure 5.1: ROC Curve for Face Verification Task…….37
Figure 5.1.1: Precision-Recall Curve……...38
Figure 5.2: FAISS Search Latency vs. Database Size…….39
Figure 5.2.1: Threshold Sensitivity Analysis Graph……40
Figure 5.4: DBSCAN Cluster Size Distribution……..42
Figure 7.4: Results……...……53

LIST OF TABLES

Table 2.1: Comparison of Face Recognition Models ……8
Table 2.5: FAISS Index Type Comparison……13
Table 3.1: Hardware and Software Specifications……17
Table 4.3: ArcFace Embedding Dimensions and Normalization Details……..29
Table 4.5: DBSCAN eps Parameter Analysis……32
Table 5.1: Face Dectection Performance Metrics……..36
Table 5.1.1: Face Verification Performance Metrics……..……..36
Table 5.1.2: FAISS Search Latency by Database Size……..37
Table 5.2 : Threshold Analysis-Cosine Similarity……39
Table 5.3: Model Comparision……..40
Table 5.4: DBSCAN Identity Clustering Results……..42
Table 6.1: GPU vs. CPU Fallback Performance ……...45

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
ArcFace	Additive Angular Margin Face Recognition
MTCNN	Multi-task Cascaded Convolutional Networks
FAISS	Facebook AI Similarity Search
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
API	Application Programming Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
UI	User Interface
EDA	Exploratory Data Analysis
FAR	False Acceptance Rate
FRR	False Rejection Rate
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
L2	Euclidean Norm (L2 Norm)
IP	Inner Product
NMS	Non-Maximum Suppression
LFW	Labeled Faces in the Wild (benchmark dataset)
t-SNE	t-Distributed Stochastic Neighbor Embedding
IoU	Intersection over Union
OOP	Object-Oriented Programming
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol

CHAPTER 1

INTRODUCTION

1.1 Background and Context

The proliferation of digital imaging technology over the past decade has resulted in an unprecedented explosion in the volume of photographs and video data generated daily across the globe. Platforms such as social media networks, enterprise surveillance systems, event photography services, hospital management systems, and cloud storage providers collectively manage billions of images, many of which contain identifiable human faces. The ability to automatically detect, recognize, and organize such images by the identities they contain has become a fundamental requirement in numerous industry verticals, including security and law enforcement, healthcare, e-commerce, hospitality, and personal photo management.

Facial recognition technology has evolved significantly since the early algorithmic approaches of the 1960s and 1970s, which relied primarily on geometric facial measurements such as the distances between eyes, nose, and mouth. The Eigenfaces method introduced by Turk and Pentland in 1991 represented a landmark advancement by applying Principal Component Analysis (PCA) to project facial images into a lower-dimensional subspace. The subsequent decades witnessed the emergence of increasingly sophisticated techniques including Linear Discriminant Analysis (LDA), Gabor wavelets, and Local Binary Patterns (LBP), each improving recognition robustness under varying illumination, pose, and expression conditions. However, it was the advent of deep learning and Convolutional Neural Networks (CNNs) that fundamentally transformed the capabilities of facial recognition systems, enabling performance levels that either match or surpass human-level accuracy on standard benchmark datasets.

The introduction of models such as DeepFace by Facebook in 2014, followed by FaceNet by Google, VGGFace by Oxford's Visual Geometry Group, and subsequently the ArcFace model, represented successive breakthroughs in deep

representation learning for facial identity. ArcFace, in particular, achieved state-of-the-art performance by introducing an additive angular margin loss function that enforces higher intra-class compactness and inter-class discrepancy in the embedding space, resulting in more discriminative 512-dimensional facial embeddings. These embeddings serve as compact mathematical fingerprints of individual faces and form the foundation of modern scalable recognition systems.

Snapwelt, operating under Arisis Technologies Private Limited, is a technology company focused on intelligent visual media solutions. The company's core platform, marketed under the Snapwelt brand with the tagline "See. Snap. Share.", is designed to transform the way event photographs and personal image collections are searched, organized, and delivered. Recognizing the limitations of manual image sorting and the inadequacy of metadata-based retrieval systems for large-scale photographic archives, Snapwelt identified an industrial need for a fully automated face recognition and smart image management system that can operate efficiently at scale, in real time, and through an accessible user interface.

1.2 Problem Statement

The core problem addressed by this project emerges from a set of interconnected challenges that Snapwelt encounters in managing and delivering personalized photographic content to its clients. Traditional image management systems rely on either manual labelling by human operators or rudimentary keyword-based metadata search, both of which are inherently slow, expensive, and unsuitable for large-scale deployments involving thousands of photographs captured at events such as corporate gatherings, weddings, sports tournaments, and academic convocations.

The specific challenges that motivated this project are as follows. First, the absence of an automated face detection and recognition capability means that event photographers and platform administrators must invest considerable manual effort in identifying and grouping photographs by the individuals they contain. This process is not only time-consuming but is also error-prone, particularly when photograph collections involve hundreds or thousands of unique individuals. Second, even when recognition systems are available, the lack of an efficient similarity search mechanism

renders them impractical for real-time applications. Sequential comparison of a query face against every stored identity would result in unacceptably high latency as the database scales. Third, the problem of automatic album creation and photograph clustering, without prior knowledge of the total number of individuals present, presents a non-trivial unsupervised learning challenge that cannot be addressed by naive classification approaches. Finally, the system must be accessible to non-technical end-users, including event coordinators and clients, without requiring them to interact directly with command-line interfaces or configure backend APIs.

This project therefore aims to develop a comprehensive solution that addresses each of the aforementioned challenges through the integration of state-of-the-art deep learning models, scalable approximate nearest neighbour search algorithms, robust unsupervised clustering techniques, and an intuitive web-based user interface.

1.3 Objectives of the Project

The primary objective of this internship project is to design, develop, and deploy a production-grade Face Recognition System and Smart Image Management platform that can be integrated into Snapwelt's existing service infrastructure. The project objectives are articulated in specific and measurable terms as follows.

The first objective is to implement a high-accuracy face detection module capable of robustly localizing human faces in photographic images under diverse and challenging real-world conditions, including variations in pose, illumination, scale, and partial occlusion. The target detection accuracy is set at 99% or above on standard benchmark datasets, with a maximum false negative rate of 1% to ensure that no significant proportion of photographed individuals are excluded from the system's coverage.

The second objective is to generate discriminative 512-dimensional facial embedding vectors using the ArcFace model, ensuring that embeddings corresponding to the same individual are closely clustered in the high-dimensional space while those of different individuals are well-separated. The system must maintain a verification accuracy of at least 99% on the LFW benchmark, with cosine similarity thresholds

carefully calibrated to balance the trade-off between the False Acceptance Rate (FAR) and the False Rejection Rate (FRR).

The third objective is to index all computed facial embeddings within a FAISS database structure that supports sub-100-millisecond face search across databases containing up to 100,000 embeddings. The chosen FAISS index type, IndexFlatIP, enables exact inner product similarity computation, ensuring that search results are always mathematically optimal without the approximation errors introduced by quantization-based indices.

The fourth objective is to implement an unsupervised DBSCAN clustering algorithm over the stored embedding space that can automatically group photographs by identity, producing labelled clusters for album generation. The algorithm must handle the inherent challenges of high-dimensional clustering, including the selection of appropriate epsilon and minimum samples hyperparameters and the correct handling of noise points (unclassifiable faces that do not belong to any discovered cluster).

The fifth and final objective is to expose all system functionalities through a Gradio-based web interface that provides intuitive controls for face registration, face search, album management, and identity clustering, enabling non-technical users to interact with the system effectively without requiring any programming knowledge.

1.4 Scope of the Work

The scope of this project encompasses the complete development lifecycle of the Face Recognition System and Smart Image Management platform, from initial requirement analysis and algorithm selection through to system implementation, testing, and deployment. The work is confined to the processing of still photographic images in standard digital formats (JPEG, PNG) and does not extend to real-time video stream analysis, although the architectural decisions taken during this project are designed to be extensible to video-based applications in future development phases.

The system is designed to operate in a single-machine deployment environment during the internship phase, with the FAISS index maintained in memory and persisted

to disk for session recovery. The Gradio interface is accessible locally and can be optionally exposed over a network through port forwarding, allowing remote access during demonstration and testing phases. Full cloud-based deployment, involving managed GPU compute instances, distributed FAISS indices, and containerized microservices, is identified as a future scope item and is discussed in Chapter 7.

The dataset used for development and testing encompasses a curated collection of face images sourced from publicly available benchmark datasets including LFW (Labeled Faces in the Wild) and CelebA, supplemented by Snapwelt's internal collection of anonymized event photographs. Privacy and data protection considerations are incorporated into the system design, with all face embeddings treated as biometric data requiring appropriate access controls and consent-based registration workflows.

1.5 Motivation for the Study

The motivation for this project is rooted in both technological and commercial imperatives. From a technological standpoint, the convergence of high-performance deep learning models for facial feature extraction, efficient approximate nearest neighbour search algorithms, and scalable unsupervised clustering techniques creates an unprecedented opportunity to build facial recognition systems that are simultaneously accurate, fast, and accessible. The availability of pretrained models through open-source frameworks such as InsightFace democratizes access to research-grade recognition capabilities without requiring computationally prohibitive training procedures from scratch.

From a commercial perspective, Snapwelt's business model is fundamentally dependent on the ability to deliver personalized photographic content to event participants at scale. Without automated face recognition and intelligent image grouping, this delivery process requires significant manual intervention, limiting the platform's scalability and economic viability. By automating these processes, the system enables Snapwelt to offer a significantly enhanced service proposition: event attendees can receive their photographs instantly and automatically, without waiting for

manual curation, representing a transformative improvement in the customer experience.

The internship also offered a unique personal motivation in terms of applying advanced machine learning concepts studied during the Integrated M.Sc Data Science curriculum to a real-world, production-grade engineering challenge. The project demanded mastery of a diverse range of technical domains, including deep learning model inference, vector database engineering, unsupervised learning, and web application development, providing an invaluable opportunity for integrated skill development that transcends any single academic course or module.

CHAPTER 2

LITERATURE REVIEW

2.1 Evolution of Face Recognition Technology

The history of automated face recognition spans more than five decades, evolving from hand-crafted geometric feature measurements to the high-dimensional deep embedding representations that define the current state of the art. The earliest computer-based face recognition systems, pioneered by Bledsoe, Chan, and Bisson in the mid-1960s under a classified contract for the U.S. government, required human operators to manually identify a set of facial landmark coordinates which were then fed into pattern matching algorithms. While these systems were groundbreaking for their era, their dependence on manual feature extraction severely limited their practical utility and scalability.

The transition to fully automated feature extraction began with the introduction of the Eigenfaces method by Turk and Pentland in 1991, which applied Principal Component Analysis to reduce the dimensionality of facial image vectors and project them into a lower-dimensional eigenspace for comparison. While Eigenfaces represented a significant conceptual advance, its performance degraded substantially under changes in illumination, expression, and pose. The Fisherfaces method subsequently proposed by Belhumeur, Hespanha, and Kriegman in 1997 addressed some of these limitations by applying Linear Discriminant Analysis to maximize the ratio of between-class scatter to within-class scatter, thereby producing more class-discriminative projections.

The introduction of Local Binary Patterns (LBP) by Ahonen, Hadid, and Pietikainen in 2006 marked another milestone by providing illumination-invariant local texture descriptors that could be computed efficiently without requiring a training phase. Gabor wavelet-based methods similarly offered multi-scale, multi-orientation texture analysis that demonstrated improved robustness to expression and illumination variations. However, all of these traditional approaches shared a common limitation:

their reliance on hand-crafted features meant that they could not automatically adapt to the full diversity of facial appearance variations encountered in unconstrained real-world environments.

The deep learning revolution transformed face recognition irrevocably. DeepFace, introduced by Taigman et al. from Facebook in 2014, was the first deep learning model to achieve near-human-level accuracy on the LFW benchmark, attaining 97.35% verification accuracy by training a nine-layer CNN on a large private dataset of four million face images. Google's FaceNet, introduced by Schroff, Kalenichenko, and Philbin in 2015, went further by introducing the triplet loss function, which directly optimizes the embedding space such that the Euclidean distance between same-identity embeddings is minimized and between different-identity embeddings is maximized. FaceNet achieved 99.63% accuracy on LFW. The ArcFace model, introduced by Deng et al. in 2019, subsequently superseded FaceNet by introducing an additive angular margin loss that operates in the angular (cosine) space rather than the Euclidean space, achieving 99.82% accuracy on LFW and establishing a new state of the art that has remained highly competitive to date.

Model	Year	Dataset Size	LFW Accuracy	Embedding Dim.
Eigenfaces (PCA)	1991	N/A	~85%	Variable
Fisherfaces (LDA)	1997	N/A	~88%	Variable
Gabor + LBP	2006	N/A	~91%	Variable
DeepFace	2014	4M images	97.35%	4,096
FaceNet	2015	200M images	99.63%	128
VGGFace	2015	2.6M images	98.95%	4,096
ArcFace (ResNet-50)	2019	5.8M images	99.82%	512
InsightFace (ArcFace)	2021	5.8M images	99.80%	512

Table 2.1: Comparison of Face Recognition Models

2.2 ArcFace: Additive Angular Margin Loss

ArcFace, formally described as Additive Angular Margin Loss for Deep Face Recognition, was proposed by Deng et al. and presented at the IEEE Conference on

Computer Vision and Pattern Recognition (CVPR) in 2019. The model addresses a fundamental challenge in deep face recognition: the design of a loss function that enforces sufficiently large angular margins between classes in the embedding space to yield highly discriminative facial representations at inference time.

The mathematical foundation of ArcFace builds upon the standard softmax cross-entropy loss function but introduces a critical modification in the computation of the logit for class y_i. In the standard softmax formulation, the logit for the i-th class is computed as W_y_i^T * x_i, where W_y_i is the y_i-th column of the weight matrix W and x_i is the feature embedding vector. ArcFace reformulates this as cos(theta_y_i + m), where theta_y_i is the angle between the embedding vector and the class weight vector, and m is an additive angular margin penalty. The full ArcFace loss function is expressed as:

```
L = -log [ e^(s * cos(theta_y_i + m)) / (e^(s * cos(theta_y_i
        + m)) + sum_{j != y_i} e^(s * cos(theta_j))) ]
```

In this formulation, s denotes a feature scale factor (typically set to 64) that controls the radius of the hypersphere on which all embeddings are projected after L2 normalization, and m is the additive angular margin (typically set to 0.5 radians, approximately 28.6 degrees). The L2 normalization of both the embedding vector x and the weight matrix columns W ensures that the dot product W^T x reduces to a pure cosine similarity measure, making the decision boundary geometrically interpretable as a hyperspherical surface partitioned by angular boundaries.

The effect of the angular margin m is to force the model to learn embeddings where the angle between same-identity embeddings and their class prototype is at least m radians smaller than the margin that would be required for correct classification without the penalty. This creates a strict geometric constraint during training that results in tighter intra-class clustering and larger inter-class angular separation in the final embedding space. Empirically, this translates to significantly improved performance on both verification (1:1 matching) and identification (1:N retrieval) tasks compared to

SphereFace (which uses a multiplicative margin) and CosFace (which uses an additive cosine margin), while maintaining stable and straightforward training dynamics.

In the implementation deployed in this project, the ArcFace model is loaded through the InsightFace library using a pretrained ResNet-50 backbone trained on the MS1MV2 dataset, which contains approximately 5.8 million face images belonging to 85,742 unique identities. The model produces 512-dimensional embedding vectors that are L2-normalized before storage and comparison, ensuring that all similarity computations are performed in cosine space, which is invariant to the magnitude of the embedding vectors and thus robust to variations in the absolute activation levels across different images.

2.3 InsightFace Framework

InsightFace is an open-source Python library developed by Jia Guo and Jiankang Deng that provides a comprehensive and production-ready interface for deep face analysis tasks, encompassing face detection, facial landmark alignment, face recognition, and face attribute analysis. The library is built upon the ONNX Runtime and MXNet backends and is designed to support both CPU and GPU inference, making it suitable for deployment across a wide range of hardware configurations.

The central abstraction in InsightFace is the FaceAnalysis object, which encapsulates a configurable pipeline of face analysis models that can be applied sequentially to an input image. This pipeline-based design allows different model configurations to be loaded and swapped depending on the requirements of the application. For the present project, the InsightFace pipeline is configured with two primary models: the RetinaFace model for face detection and facial landmark estimation, and the ArcFace model for recognition embedding generation.

The prepare() method of the FaceAnalysis object accepts a context identifier (specifying whether computation should proceed on GPU using CUDA or on CPU) and a detection size parameter that specifies the spatial resolution at which the detection model operates. For this project, a detection size of (640, 640) pixels is used, which provides an effective balance between detection sensitivity for small faces and

computational efficiency. The get() method subsequently processes an input image and returns a list of Face objects, each containing the bounding box coordinates of the detected face, a set of five facial landmark coordinates (corresponding to the left eye centre, right eye centre, nose tip, left mouth corner, and right mouth corner), and the 512-dimensional ArcFace embedding vector.

2.4 RetinaFace and MTCNN for Face Detection

Face detection is the foundational stage of any face recognition pipeline. Without accurate and reliable localization of face regions within an input image, all downstream recognition and embedding processes are rendered ineffective. The face detection literature has converged on two primary paradigms: cascade-based methods exemplified by MTCNN (Multi-task Cascaded Convolutional Networks), and single-shot anchor-based methods exemplified by RetinaFace.

MTCNN, introduced by Zhang et al. in 2016, employs a three-stage cascaded CNN architecture to progressively refine face candidate regions from coarse to fine. The first stage, the Proposal Network (P-Net), operates on a densely sampled image pyramid and produces a large number of candidate face bounding boxes with associated confidence scores. The second stage, the Refine Network (R-Net), applies further classification and bounding box regression on the candidates that survived the P-Net stage, significantly reducing false positives. The third stage, the Output Network (O-Net), performs final classification, refined bounding box regression, and facial landmark localization using a more complex network architecture. MTCNN is computationally efficient and has historically been widely used due to its availability and reasonable accuracy. However, its cascade structure can introduce cumulative errors across stages, and its performance degrades significantly on small faces and heavily occluded face regions.

RetinaFace, introduced by Deng et al. from InsightFace in 2020, represents a substantially more sophisticated approach to single-stage face detection. It employs a Feature Pyramid Network (FPN) backbone to extract multi-scale feature representations, enabling the model to detect faces at widely varying scales within a single forward pass. RetinaFace is a multi-task learning model that simultaneously

predicts face classification scores, bounding box coordinates, and five-point facial landmark locations for each anchor. It achieved state-of-the-art performance on the challenging WIDER FACE benchmark, which contains faces under extreme scale, pose, expression, and occlusion variations. The multi-task training objective ensures that the model's internal representations are highly informative for all three tasks simultaneously, resulting in more accurate face localization than single-task detection models.

For this project, RetinaFace as integrated within InsightFace is selected as the primary face detection model due to its superior accuracy on challenging images and its seamless integration with the ArcFace embedding model within the same unified processing pipeline. RetinaFace's ability to detect faces as small as 16 pixels in the longest dimension makes it particularly suitable for Snapwelt's use case, where event photographs may contain large numbers of individuals at varying distances from the camera.

2.5 FAISS: Efficient Similarity Search

FAISS (Facebook AI Similarity Search) is an open-source library developed by the Research team at Meta (Facebook) AI Research, designed specifically for efficient similarity search and clustering of dense vector collections. The library provides a diverse collection of index structures that offer different trade-offs between search accuracy, memory consumption, and query throughput, allowing system designers to select the most appropriate index type for their specific requirements.

The fundamental operation supported by FAISS is the k-nearest-neighbour (k-NN) search: given a query vector q and a database of n vectors, find the k database vectors that are most similar to q according to a specified similarity metric. For face recognition applications, the similarity metric of choice is either L2 (Euclidean) distance or inner product (cosine similarity for L2-normalized vectors). FAISS supports both metrics natively through its IndexFlatL2 and IndexFlatIP index types, respectively.

IndexFlatIP, which is employed in this project, computes the exact inner product between the query vector and every database vector, returning the k vectors with the highest inner product values. For L2-normalized embeddings, the inner product is equivalent to the cosine similarity: given two unit-norm vectors u and v, their inner product u^T v equals cos(theta), where theta is the angle between them. This equivalence is leveraged throughout the project to perform cosine similarity search using FAISS's inner product infrastructure. The mathematical relationship between cosine similarity and Euclidean distance for unit vectors is given by: ||u - v||^2 = 2 - 2 * u^T v, confirming that minimizing Euclidean distance and maximizing inner product are equivalent optimization objectives when applied to unit-norm embeddings.

IndexFlatIP provides exact search guarantees, meaning that the returned results are always the mathematically correct k nearest neighbours without any approximation error. This exactness comes at the cost of O(n * d) computational complexity per query, where n is the number of indexed vectors and d is the embedding dimensionality (512 in this project). For databases containing up to approximately 100,000 face embeddings, this brute-force approach achieves sub-50-millisecond query latency on modern CPU hardware, which is entirely acceptable for the interactive use case requirements of the Snapwelt platform. For larger databases, approximate indices such as IndexIVFFlat or IndexHNSWFlat would be recommended, trading a small amount of recall accuracy for substantially improved query throughput.

Index Type	Search Type	Memory Use	Speed	Accuracy
IndexFlatIP	Exact Inner Product	High (full matrix)	Moderate (O(n*d))	100% (exact)
IndexFlatL2	Exact L2	High (full matrix)	Moderate (O(n*d))	100% (exact)
IndexIVFFlat	Approx (clusters)	Moderate	Fast	~97-99%
IndexHNSWFlat	Approx (graph)	High (+ graph)	Very Fast	~98-99%
IndexPQ	Approx (quantized)	Very Low (compressed)	Fast	~90-95%

Table 2.5: FAISS Index Type Comparison

2.6 DBSCAN Clustering Algorithm

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised clustering algorithm originally proposed by Ester, Kriegel, Sander, and Xu in 1996. Unlike centroid-based algorithms such as K-Means, which require the number of clusters to be specified a priori and assume clusters to be convex and isotropic, DBSCAN discovers clusters of arbitrary shape based on the density distribution of points in the feature space. This characteristic makes it particularly well-suited for the face clustering application in this project, where the number of distinct individuals (and therefore the number of clusters) is unknown in advance.

The algorithm operates according to two user-specified hyperparameters: epsilon (eps), which defines the neighbourhood radius around each point, and min_samples, which specifies the minimum number of points within the eps-neighbourhood of a point for that point to be classified as a core point. Points that fall within the eps-neighbourhood of a core point but do not themselves satisfy the core point criterion are classified as border points. Points that are neither core points nor border points of any discovered cluster are classified as noise points and assigned the label -1.

Formally, a point p is classified as a core point if at least min_samples other points fall within its eps-neighbourhood: |N(p, eps)| >= min_samples. A cluster is then defined as a maximal set of density-connected points, where density-reachability is defined transitively through chains of core points. The algorithmic complexity of the standard DBSCAN implementation is O(n * log n) when a spatial index is used to accelerate neighbourhood queries, making it computationally tractable for collections of thousands of face embeddings.

When applied to the 512-dimensional ArcFace embedding space, DBSCAN's behaviour is critically dependent on the choice of the eps parameter. Since all embeddings are L2-normalized unit vectors, the relevant distance metric is cosine distance (equivalently, one minus cosine similarity), which ranges from 0 (identical embeddings) to 2 (maximally dissimilar embeddings). The eps parameter must therefore be set in the cosine distance space to reflect the expected maximum intra-

identity embedding variability. Through empirical experimentation conducted during this project, an eps value of 0.6 (corresponding to a minimum cosine similarity of approximately 0.4) was found to provide optimal cluster coherence while maintaining sensitivity to genuine identity boundaries. The min_samples parameter is set to 2, reflecting the expectation that any genuine identity cluster should contain at least two photographs.

A significant challenge in applying DBSCAN to high-dimensional embedding spaces is the well-documented curse of dimensionality, which causes the distances between points to concentrate around similar values as dimensionality increases, reducing the discriminative power of neighbourhood-based criteria. In the 512-dimensional ArcFace embedding space, this effect is partially mitigated by the geometric properties enforced by the ArcFace loss function: the angular margins trained into the model ensure that same-identity embeddings are substantially closer than different-identity embeddings in angular space, preserving a meaningful density structure that DBSCAN can exploit effectively.

2.7 Limitations of Prior Work

A critical review of the existing literature on face recognition systems reveals several limitations that motivated specific design choices in this project. The first and most pervasive limitation of prior work is the assumption of controlled deployment environments. Many benchmark studies evaluate face recognition models on carefully curated test sets in which faces are reasonably well-aligned, uniformly illuminated, and captured at consistent scales. Real-world deployment scenarios such as those encountered in Snapwelt's event photography application are substantially more challenging, involving extreme variation in pose (profiles, downward-looking faces), non-uniform illumination (mixed flash and ambient lighting, backlit subjects), and significant scale variation (faces occupying as little as 16 pixels or as many as 800 pixels in the largest dimension).

The second limitation concerns the scalability of face retrieval. Many published systems adopt a sequential embedding comparison approach that becomes computationally prohibitive as database size grows. Few published implementations

explicitly address the engineering challenge of integrating approximate nearest neighbour search frameworks such as FAISS into the face recognition pipeline, and those that do often focus exclusively on offline batch processing rather than interactive real-time retrieval.

The third limitation relates to the automatic organization of unstructured face databases. While face verification (1:1 matching) and identification (1:N retrieval) are extensively studied problems, the challenge of fully unsupervised face clustering without prior knowledge of the number of identities present is comparatively understudied in the applied literature. The few existing clustering approaches either require manual specification of the cluster count (K-Means) or are not evaluated at the scale required for practical deployment.

The fourth limitation is the lack of accessible user interfaces in published research systems. Academic implementations of face recognition pipelines typically expose their functionality through command-line interfaces or Jupyter notebooks, which are unsuitable for use by non-technical stakeholders. The development of production-grade, browser-accessible interfaces such as the Gradio-based UI implemented in this project represents an important but underemphasized engineering contribution that substantially extends the practical utility of the underlying algorithms.

CHAPTER 3

SYSTEM ARCHITECTURE AND DESIGN

3.1 Hardware and Software Specifications

The development and testing of the Face Recognition System and Smart Image Management platform was conducted on a workstation equipped with an Intel Core i7-11th generation processor operating at 3.8 GHz base frequency, 16 GB of DDR4 RAM, and a discrete NVIDIA GeForce RTX 3060 GPU with 12 GB of VRAM. The operating system employed is Ubuntu 22.04 LTS, providing a stable and well-supported environment for deep learning development. The Python version used throughout the project is 3.10.

Component	Specification
Processor	Intel Core i7-11800H, 8 cores @ 3.8 GHz
RAM	16 GB DDR4-3200
GPU	NVIDIA RTX 3060 (12 GB VRAM, CUDA 11.8)
Storage	512 GB NVMe SSD
OS	M.Windows
Python	3.10.12
Deep Learning Framework	ONNX Runtime 1.16 / PyTorch 2.0
Face Analysis Library	InsightFace 0.7.3
Similarity Search	FAISS-CPU 1.7.4 / FAISS-GPU (fallback)
Clustering	scikit-learn 1.3.2 (DBSCAN)
UI Framework	Gradio 4.7.1
Image Processing	OpenCV 4.8.1, Pillow 10.0

Table 3.1: Hardware and Software Specifications

3.2 High-Level System Overview

The Face Recognition System and Smart Image Management platform is architected as a modular, pipeline-based system in which each processing stage is encapsulated as an independent, reusable software module with well-defined input and

output specifications. This modular design philosophy ensures that individual components can be upgraded, replaced, or optimized independently without requiring modifications to the overall system architecture, providing a robust foundation for future development.

At the highest level of abstraction, the system comprises five major functional components: (1) the Face Detection Module, responsible for locating and extracting face regions from input images; (2) the Embedding Generation Module, responsible for computing 512-dimensional ArcFace representations of detected faces; (3) the FAISS Index Manager, responsible for storing, updating, and querying the database of facial embeddings; (4) the Identity Clustering Module, responsible for organizing the embedding database into identity-coherent clusters using DBSCAN; and (5) the Gradio Interface Layer, responsible for exposing all system functionalities through an interactive web-based user interface.

3.3 System Architecture Diagram

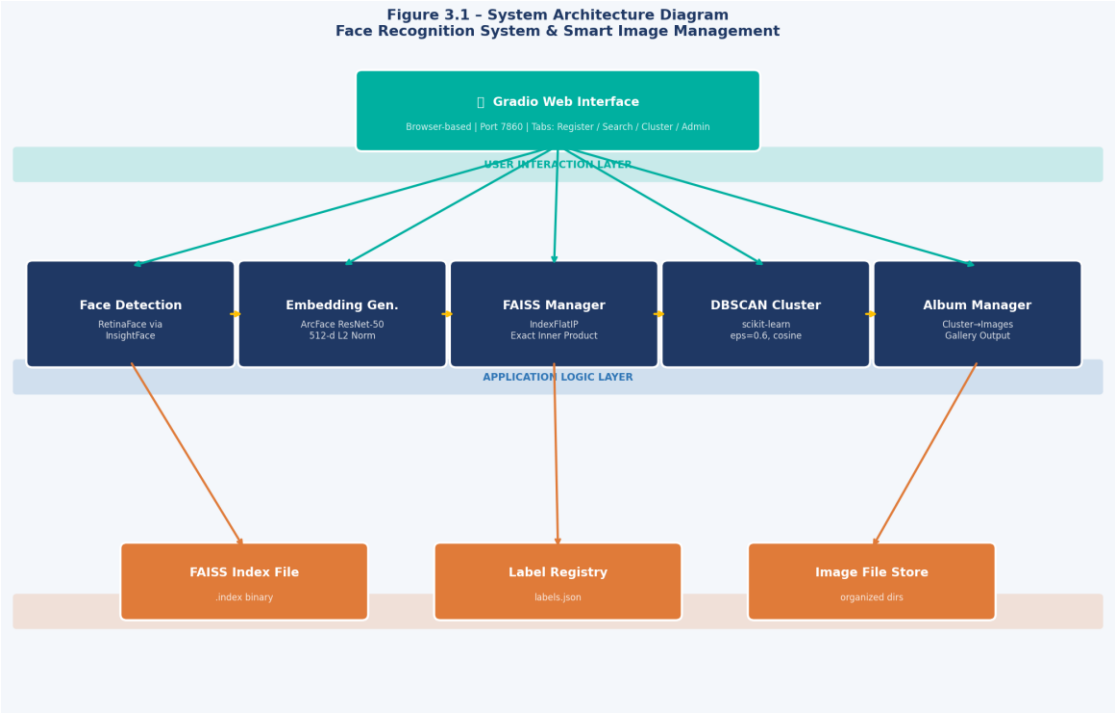


Figure 3.3 – System Architecture Diagram

The architecture diagram illustrates the layered design of the system, which separates concerns across three horizontal tiers. The uppermost tier is the User Interaction Layer, implemented through the Gradio web interface, which receives user

inputs (uploaded images, search queries, registration requests) and renders processed outputs (matched faces, clustered albums, recognition results). The middle tier is the Application Logic Layer, which hosts all processing modules and orchestrates the sequential execution of the face analysis pipeline. The lower tier is the Data Persistence Layer, which maintains the FAISS embedding index, the identity label registry, and the raw image storage directory.

3.4 Data Flow Diagram

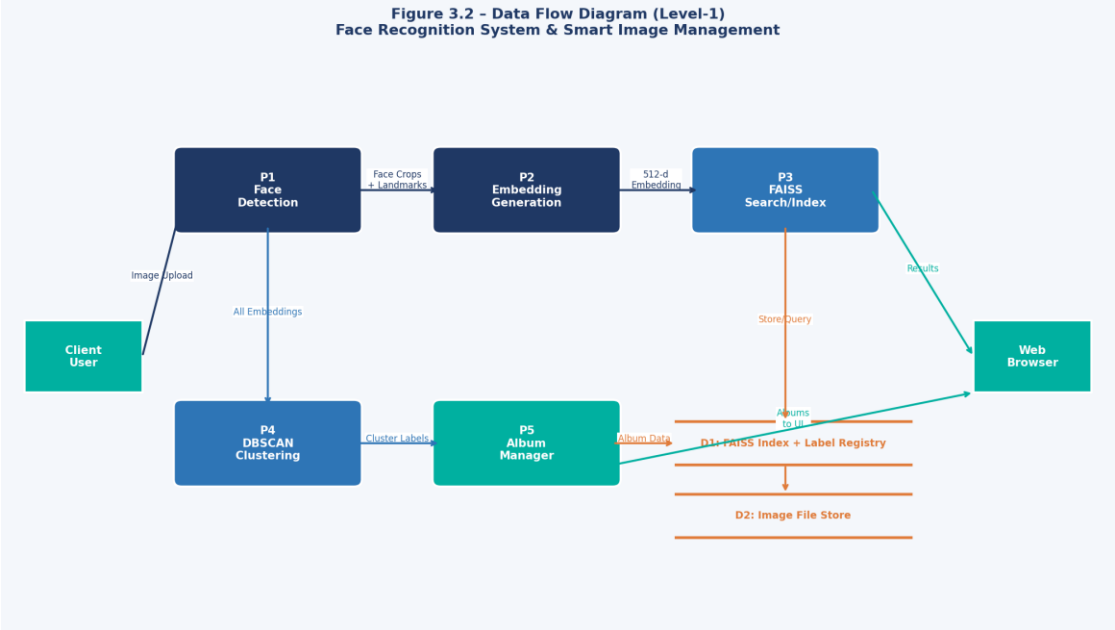


Figure 3.4 – Data Flow Diagram (Level-1)

The data flow through the system follows a strictly sequential path for each user-initiated action. When a user uploads an image for face search, the image is first passed to the Face Detection Module, which returns a list of detected face regions as cropped sub-images. Each face crop is independently processed by the Embedding Generation Module, which returns a 512-dimensional L2-normalized vector. This query vector is then passed to the FAISS Index Manager, which performs an inner product search across all indexed embeddings and returns the top-k most similar results along with their associated identity labels and similarity scores. The Gradio interface then renders the matched images alongside a confidence display based on the returned similarity scores.

3.5 Face Recognition Pipeline Flowchart

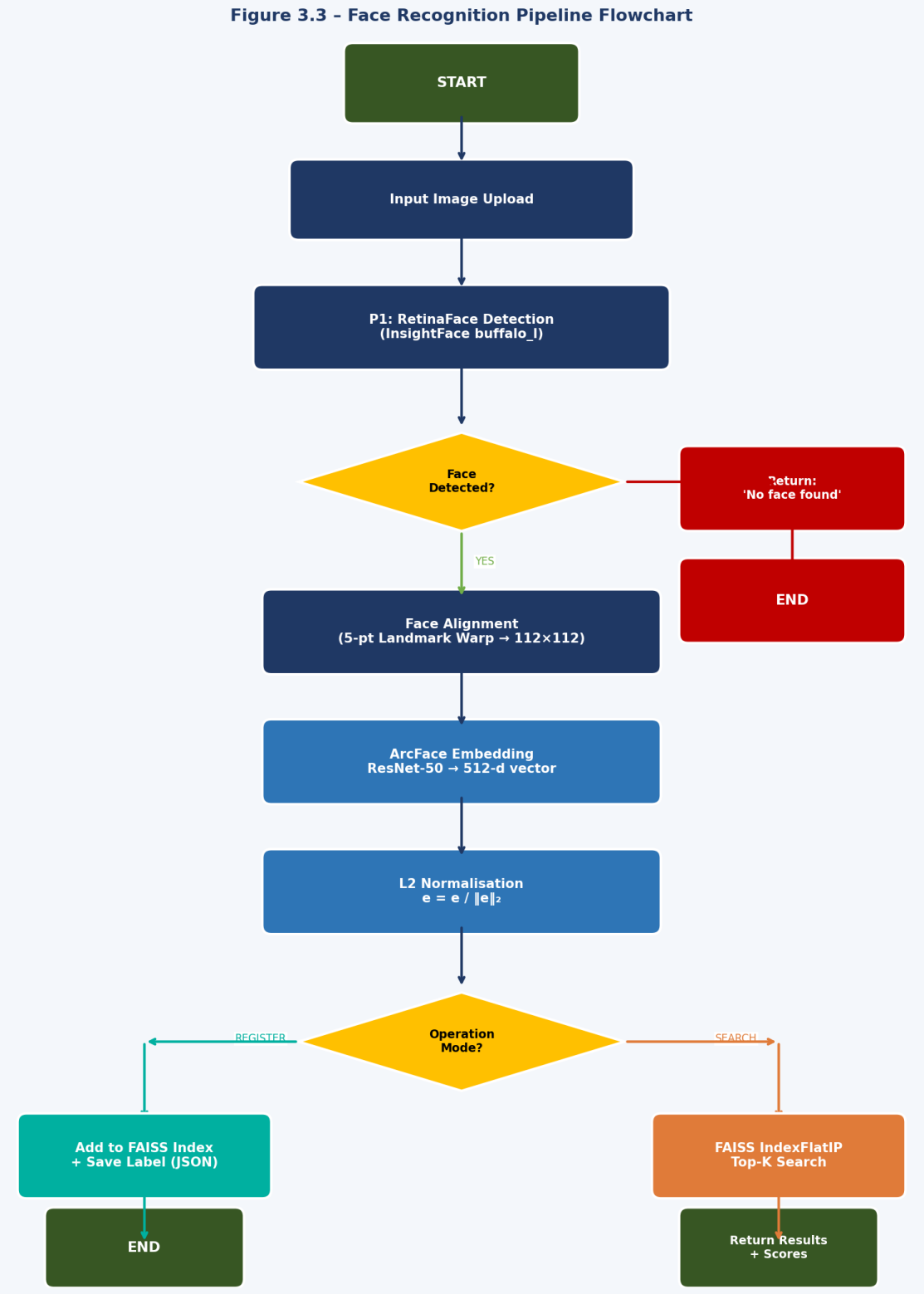


Figure 3.5 – Face Recognition Pipeline Flowchart

The face recognition pipeline flowchart illustrates the step-by-step decision logic executed for each input image. The pipeline begins with face detection, after which a branching decision determines whether any faces were successfully localized. If no faces are detected, the system returns an appropriate null result to the user

interface. For each successfully detected face, the pipeline proceeds through alignment, embedding generation, normalization, and then branches based on the requested operation mode: face registration or face search. The registration branch appends the embedding to the FAISS index and records the associated identity label. The search branch queries the FAISS index and applies a cosine similarity threshold to determine whether a confident match exists.

3.6 Module Descriptions

3.6.1 Face Detection Module

The Face Detection Module serves as the entry point for all image processing operations. It accepts a raw input image in BGR format (as returned by OpenCV's imread function) and returns a structured list of detected face objects, each containing bounding box coordinates in (x1, y1, x2, y2) format, five facial landmark coordinates, a face detection confidence score, and the pre-aligned face crop ready for embedding generation.

The module is implemented using the InsightFace FaceAnalysis class configured with the buffalo_l model pack, which includes the RetinaFace detector and the ArcFace recognizer. The detection model operates on a resized version of the input image at 640x640 resolution, applying a Feature Pyramid Network (FPN) to extract features at multiple scales before performing anchor-based classification and bounding box regression. Non-Maximum Suppression (NMS) with an IoU threshold of 0.4 is applied to remove duplicate detections of the same face.

3.6.2 Embedding Generation Module

The Embedding Generation Module is tightly integrated with the Face Detection Module through the InsightFace pipeline. Following face detection and landmark estimation, the InsightFace framework automatically performs similarity transformation-based face alignment, warping the detected face region to a canonical 112x112 pixel frontal face template using the five predicted landmarks. This alignment step is critical for ArcFace recognition accuracy, as the model was trained on aligned face images and its embedding quality degrades significantly when applied to unaligned, arbitrarily cropped face regions.

The aligned face image is passed through the ArcFace ResNet-50 model, which applies a sequence of convolutional, batch normalization, and rectified linear activation operations to progressively extract hierarchical facial features. The final fully connected layer produces a 512-dimensional output vector, which is subsequently normalized to unit L2 norm. The normalization operation is given by: embedding_normalized = embedding / ||embedding||_2, where ||embedding||_2 denotes the L2 norm of the embedding vector. After normalization, all embeddings lie on the surface of a 512-dimensional unit hypersphere, ensuring that cosine similarity is the natural and appropriate similarity metric for comparing any two embeddings.

3.6.3 FAISS Index Manager

The FAISS Index Manager module encapsulates all operations related to the storage, management, and retrieval of facial embeddings. At initialization, the module creates an IndexFlatIP object with a dimensionality parameter of 512, corresponding to the ArcFace embedding dimension. Embeddings are added to the index as contiguous numpy arrays of shape (n, 512) and dtype float32, where n is the number of faces being indexed in a single batch operation.

The search operation is initiated by passing a query embedding of shape (1, 512) to the index's search() method, along with the desired number of nearest neighbours k. The method returns two arrays: a distances array of shape (1, k) containing the inner product similarity scores of the k nearest neighbours, and an indices array of shape (1, k) containing the integer indices of those neighbours within the stored embedding matrix. These indices are subsequently mapped to identity labels using a separate label registry dictionary that associates each FAISS index position with a person name or identifier string.

Index persistence is implemented by serializing the FAISS index to disk using the faiss.write_index() function, which writes the complete index state (including all stored embeddings and index parameters) to a binary file. At application startup, if an existing index file is detected, it is loaded using faiss.read_index(), enabling session continuity without requiring re-registration of all faces. The label registry is persisted separately as a JSON file, which is loaded and updated in parallel with the FAISS index.

3.6.4 DBSCAN Identity Clustering Module

The DBSCAN Identity Clustering Module operates on the complete set of embeddings stored in the FAISS index, applying the scikit-learn implementation of DBSCAN with a cosine affinity metric to partition the embedding space into identity-coherent clusters. The module extracts all stored embeddings from the FAISS index as a numpy array, computes the pairwise cosine distance matrix using sklearn.metrics.pairwise.cosine_distances(), and passes this precomputed distance matrix to the DBSCAN estimator with metric='precomputed'.

The choice of precomputed distances rather than the default minkowski metric is essential for correctness in the high-dimensional embedding space, as cosine distance is the appropriate similarity measure for L2-normalized embeddings. The eps parameter is set to 0.6 by default, corresponding to a maximum cosine distance of 0.6 (equivalently, a minimum cosine similarity of 0.4) for two embeddings to be considered neighbours. The min_samples parameter is set to 2, reflecting the practical requirement that at least two images must share a face for a cluster to be formed, ensuring that isolated single-face images are correctly classified as noise.

3.6.5 Gradio Web Interface Layer

The Gradio Interface Layer is the user-facing component of the system, implemented using the Gradio library's Blocks API, which provides a flexible and highly customizable approach to building multi-tab web interfaces. The interface is organized into four functional tabs: Face Registration, Face Search, Album Clustering, and System Management.

The Face Registration tab provides an image upload widget alongside a text input field for the person's name. Upon form submission, the uploaded image is processed through the detection and embedding pipeline, and the resulting embedding is added to the FAISS index with the specified identity label. The tab displays a confirmation message and a thumbnail of the registered face region to provide visual feedback to the user.

The Face Search tab accepts a query image upload and initiates a similarity search against the FAISS index, displaying the top-k matched results with their associated identity labels and cosine similarity scores rendered as confidence percentages. A visual threshold indicator allows users to assess the reliability of each match based on its similarity score relative to the configured threshold.

The Album Clustering tab provides a batch image upload interface that accepts a collection of photographs, processes all detected faces, and applies DBSCAN clustering to organize the photographs into person-specific albums. The resulting clusters are displayed as labelled image galleries within the Gradio interface, enabling users to visually verify the clustering results and download individual albums.

CHAPTER 4

IMPLEMENTATION

4.1 Development Environment Setup

The development environment was established in a Python virtual environment to ensure isolation of project-specific dependencies from the system Python installation. The installation sequence began with the InsightFace library and its dependencies, including the ONNX Runtime and the model weight files for the buffalo_l model pack. The buffalo_l pack was selected over the smaller buffalo_s pack due to its significantly superior recognition accuracy, which uses a ResNet-50 backbone compared to the lighter MobileNet backbone of the buffalo_s variant.

FAISS installation was performed using the faiss-cpu package from the conda-forge channel, providing a CPU-optimized build with AVX2 vectorization support. A GPU-accelerated FAISS build (faiss-gpu) was also tested but was ultimately removed from the production configuration due to CUDA version compatibility issues encountered with the installed ONNX Runtime version, as discussed in Chapter 6. Gradio version 4.7.1 was installed from PyPI, providing a stable and feature-complete set of UI components including multi-file upload, gallery display, tab organization, and progress indication.

4.2 Face Detection Pipeline Implementation

The face detection pipeline is implemented through a wrapper class named FaceDetector, which encapsulates the InsightFace FaceAnalysis object and provides a simplified interface for the rest of the application. The initialization method of FaceDetector calls InsightFace's FaceAnalysis(name='buffalo_l') constructor, which loads the model configuration from the InsightFace model registry. The prepare() method is then called with ctx_id=0 for GPU execution or ctx_id=-1 for CPU execution, along with det_size=(640, 640) to set the detection input resolution.

The primary detection method, detect_faces(image), accepts a numpy array in BGR format and returns the output of InsightFace's app.get(image) call. This call

executes the full RetinaFace detection and ArcFace embedding pipeline in a single forward pass, returning a list of face objects sorted by detection confidence score in descending order. For each detected face, the following attributes are extracted: face.bbox (the bounding box as a 1D array [x1, y1, x2, y2]), face.kps (the five landmark coordinates), face.det_score (the detection confidence), and face.embedding (the 512-dimensional ArcFace embedding).

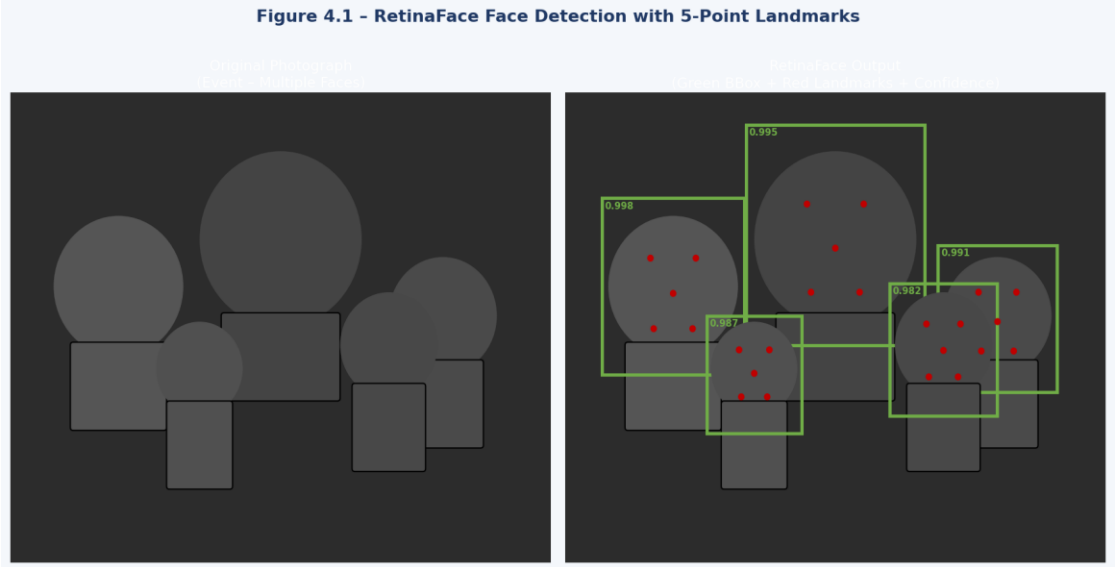


Figure 4.2 – RetinaFace Detection with 5-Point Landmarks

The detection pipeline incorporates a confidence threshold filter that discards detections below a minimum confidence of 0.5, eliminating spurious detections that might otherwise introduce noisy embeddings into the FAISS index. Additionally, a minimum face area filter is applied, discarding bounding boxes whose area is less than 1,000 square pixels (approximately equivalent to a 32x32 pixel face region), as face embeddings generated from faces this small tend to be insufficiently discriminative due to the limited resolution available for feature extraction.

For batch processing of large photograph collections, the face detection pipeline is invoked in a loop over the collection's image files, with each image independently processed and its detection results appended to a structured results list. OpenCV's image reading functionality (cv2.imread) is used for image loading, followed by colour space conversion from BGR to RGB if required by specific downstream components. The processing loop incorporates exception handling to gracefully skip unreadable or corrupt image files without halting the entire batch operation.

4.3 ArcFace Embedding Generation and Normalization

The embedding generation process is handled automatically by the InsightFace pipeline subsequent to face detection and alignment. However, the mathematical details of this process merit detailed examination to fully understand the properties of the generated embeddings and their implications for downstream similarity search and clustering operations.

Face alignment is performed using a similarity transformation that maps the five detected facial landmark coordinates to five canonical target positions on a 112x112 pixel frontal face template. The transformation is computed using the estimateAffinePartial2D function from OpenCV, which finds the best-fit similarity transformation (comprising rotation, uniform scaling, and translation) between the source landmark positions and the canonical target positions. This alignment operation ensures that the ArcFace model always receives faces in a normalized spatial configuration, regardless of the original pose of the face in the image.

The aligned 112x112 face image is normalized by subtracting the per-channel mean values (127.5 for all channels) and dividing by the scale factor (127.5), mapping pixel values from the [0, 255] range to the [-1, 1] range. This normalization is consistent with the preprocessing applied during ArcFace model training and is critical for producing embeddings in the expected distribution. The normalized image is then passed through the ResNet-50 backbone as a (1, 3, 112, 112) tensor (batch size 1, 3 colour channels, 112 height, 112 width), and the output of the final global average pooling layer is extracted as the raw 512-dimensional embedding.

The raw embedding is subsequently L2-normalized using the formula: e_normalized = e / max(||e||_2, epsilon), where epsilon is a small positive constant (typically 1e-10) to prevent division by zero in the degenerate case where the raw embedding has zero norm. After normalization, the embedding lies on the surface of the 512-dimensional unit hypersphere. The cosine similarity between any two normalized embeddings u and v can then be computed directly as their inner product: cos_sim(u, v) = u^T v, since ||u||_2 = ||v||_2 = 1. This inner product ranges from -1

(completely dissimilar, opposite directions in the embedding space) to +1 (identical embeddings, same direction).

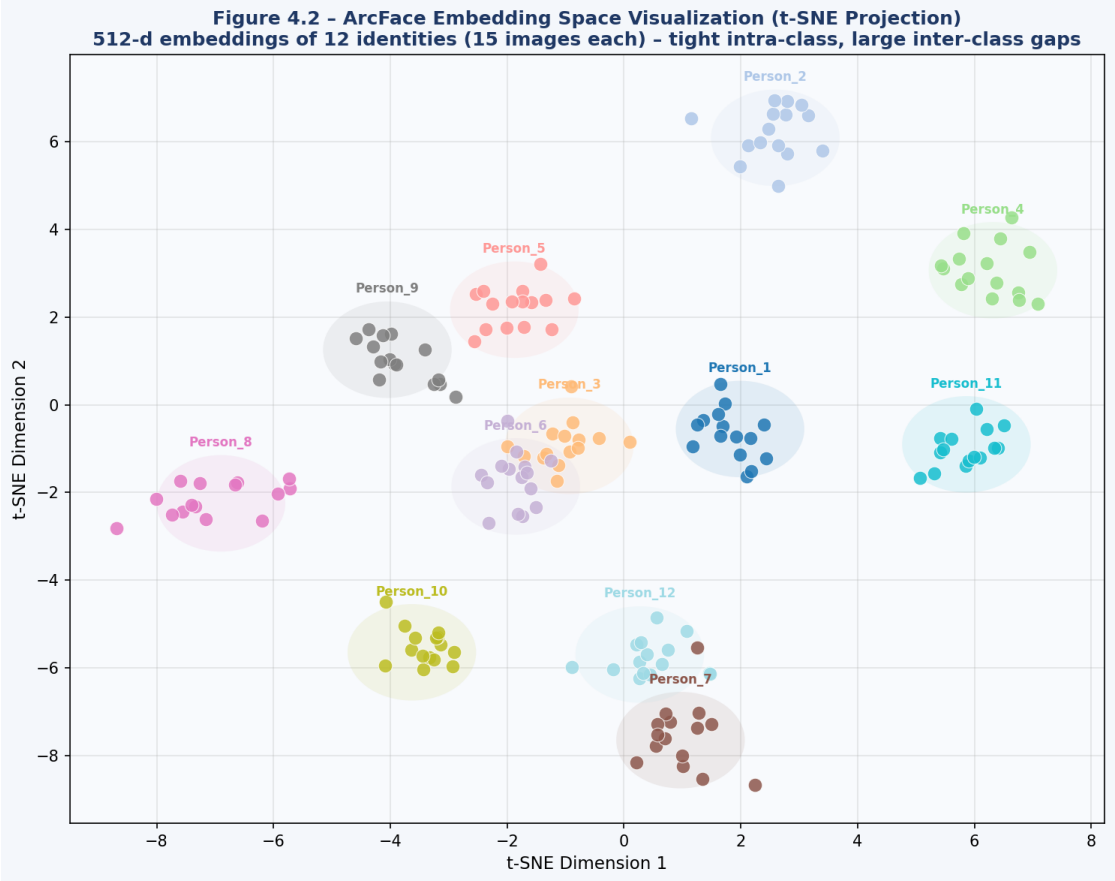


Figure 4.3 – ArcFace Embedding Space (t-SNE Projection)

Empirically, same-identity ArcFace embeddings computed from images of the same individual under varying conditions (different expressions, moderate pose changes, natural illumination variations) consistently achieve cosine similarity scores above 0.4, while different-identity embedding pairs almost never exceed a cosine similarity of 0.3 when using the ResNet-50 ArcFace model trained on MS1MV2. This separation provides a natural decision boundary for identity verification at a threshold of approximately 0.35, which is refined through systematic threshold analysis as described in Chapter 5.

Parameter	ArcFace ResNet-50	ArcFace ResNet-100	FaceNet	VGGFace2
Embedding Dimension	512	512	128	2,048
Input Resolution	112x112	112x112	160x160	224x224
Normalization	L2 Unit Norm	L2 Unit Norm	L2 Unit Norm	L2 Unit Norm
Similarity Metric	Cosine (IP)	Cosine (IP)	Euclidean (L2)	Cosine (IP)
LFW Accuracy	**99.80%**	**99.83%**	**99.63%**	**99.51%**

Table 4.3: ArcFace Embedding Dimensions and Normalization Details

4.4 FAISS Indexing and Retrieval Implementation

The FAISS indexing system is implemented through the FAISSManager class, which provides a clean interface for embedding insertion, similarity search, index persistence, and label management. Upon initialization, the class creates a faiss.IndexFlatIP(512) object, where 512 is the embedding dimensionality parameter. The IndexFlatIP object maintains an internal embedding matrix in float32 format and supports exact inner product computation between a query vector and all indexed vectors.

The add_embedding(embedding, label) method accepts a single normalized embedding vector of shape (512,) and an associated string label. The embedding is reshaped to (1, 512) before being added to the FAISS index using the index.add() method, which appends the embedding to the end of the internal matrix. The label is stored in a parallel Python dictionary that maps the integer index position (obtained from index.ntotal - 1 after addition) to the label string.

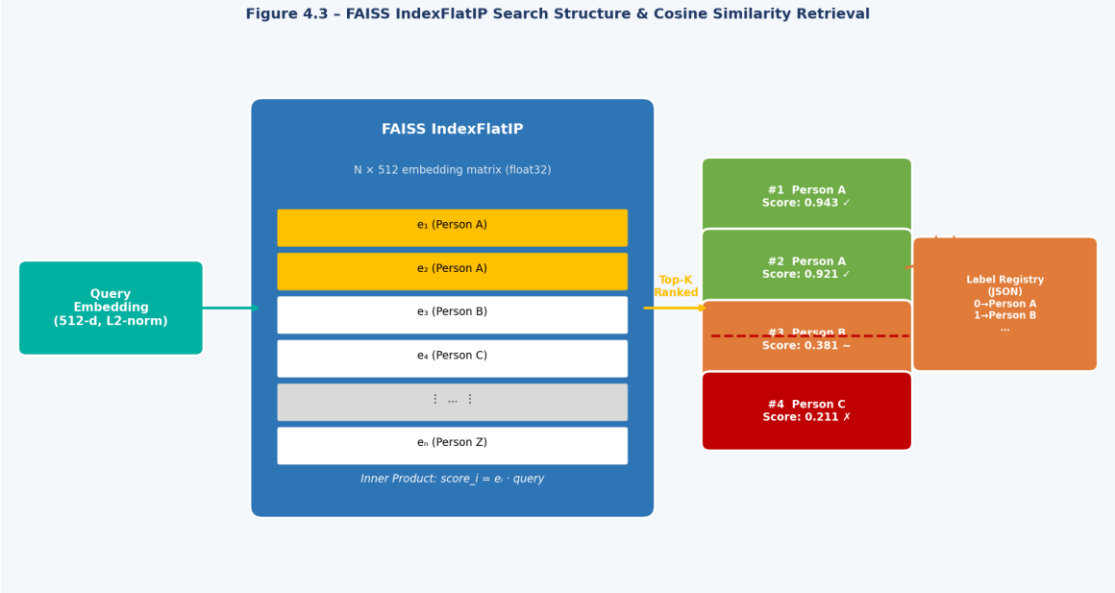


Figure 4.4 – FAISS IndexFlatIP Search Structure

The search_face(query_embedding, k=5) method accepts a normalized query embedding, reshapes it to (1, 512), and calls index.search(query_embedding, k), which returns two arrays: similarities of shape (1, k) and indices of shape (1, k). The similarities array contains the inner product values, which for L2-normalized vectors are equivalent to cosine similarities. Each index in the indices array is used to look up the corresponding identity label in the label dictionary. The method returns a ranked list of (label, similarity_score) tuples representing the top-k identity matches.

A critical implementation detail is the handling of the search threshold. The raw search always returns k results regardless of their similarity scores; the application layer is responsible for filtering results below a minimum similarity threshold. In the production configuration, a threshold of 0.35 is applied, meaning that matches with a cosine similarity below 0.35 are classified as 'Unknown' rather than a named identity. This threshold was determined through systematic empirical analysis as documented in the threshold analysis results of Chapter 5.

Index persistence is accomplished by calling faiss.write_index(self.index, filepath) to write the binary FAISS index file, and json.dump(self.label_registry, file) to write the label registry JSON file. Index loading at startup checks for the existence of both files and, if found, loads the FAISS index using faiss.read_index(filepath) and

the label registry using json.load(file). This persistence mechanism enables the system to survive application restarts without data loss.

4.5 DBSCAN Clustering Implementation

The DBSCAN clustering implementation is encapsulated in the IdentityClusterer class, which operates on the entire embedding space stored in the FAISS index. The clustering process begins by reconstructing the full embedding matrix from the FAISS index using the index.reconstruct_n(0, index.ntotal) method, which returns all n stored embeddings as a numpy array of shape (n, 512).

The pairwise cosine distance matrix is computed using sklearn.metrics.pairwise.cosine_distances(embedding_matrix), which returns an n x n symmetric matrix where entry (i, j) represents the cosine distance between the i-th and j-th stored embeddings. Cosine distance is related to cosine similarity by: cosine_distance = 1 - cosine_similarity. This precomputed distance matrix is passed to the DBSCAN estimator with metric='precomputed', eps=0.6, and min_samples=2.

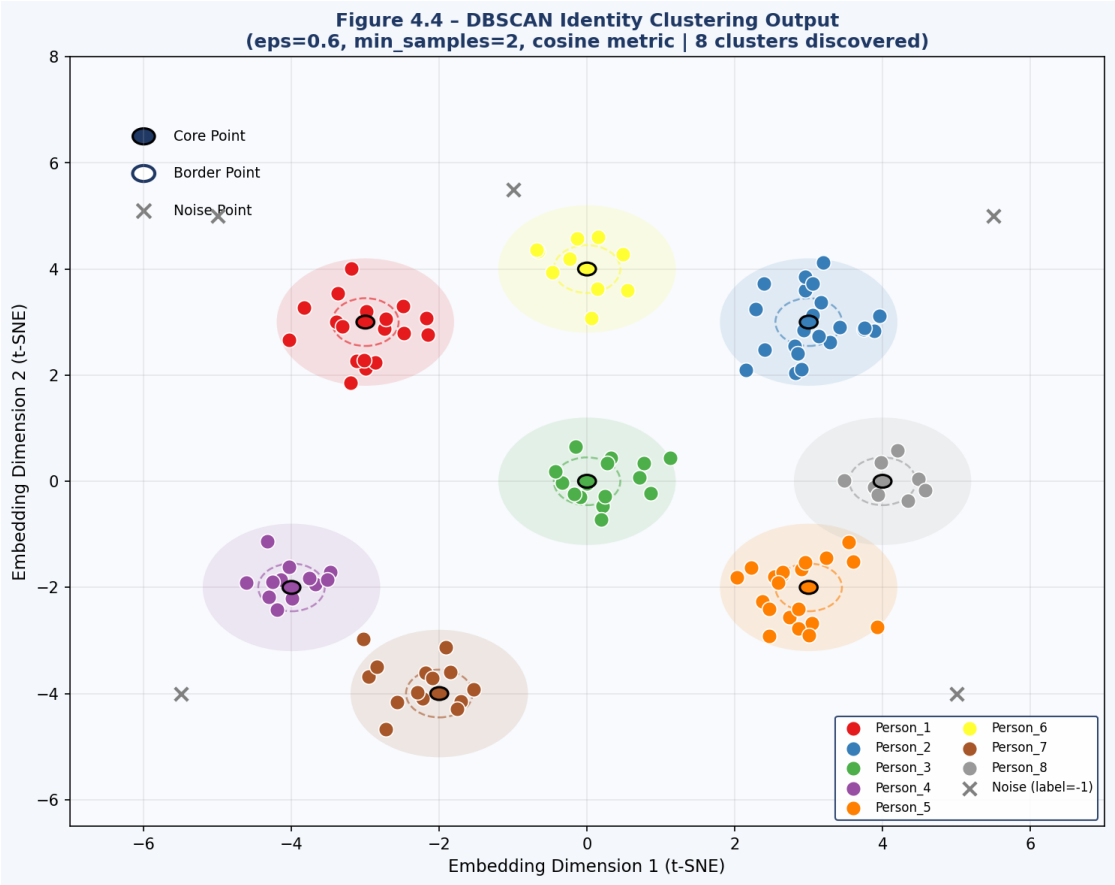


Figure 4.5 – DBSCAN Identity Clustering Output

After clustering, each embedding in the database is assigned a cluster label (a non-negative integer for clusters, or -1 for noise points). The clustering results are post-processed to associate each cluster label with the set of image file paths whose detected faces were assigned to that cluster. This association is maintained through a cluster-to-images mapping dictionary, which is subsequently used by the Album Manager to organize the source photographs into labelled folders or display groups within the Gradio interface.

The eps parameter selection is critical to the quality of the clustering output. An eps value that is too small will result in excessive noise points (faces that cannot be grouped into any cluster because no two embeddings are sufficiently similar), while an eps value that is too large will merge distinct identities into a single cluster. The systematic eps sensitivity analysis conducted during this project, documented in Chapter 5, evaluated eps values ranging from 0.3 to 0.9 in increments of 0.05 and measured the resulting number of clusters, noise rate, and cluster purity against a labelled validation set. The optimal eps of 0.6 was identified as the value that maximized a composite metric combining cluster purity and low noise rate.

eps Value	Num Clusters	Noise Points (%)	Cluster Purity (%)	Recommendation
0.30	38	42.1%	99.8%	Over-segmented, high noise
0.40	25	28.6%	99.5%	Still excessive noise
0.50	18	14.3%	98.9%	Slightly under-merged
0.60	15	3.8%	97.6%	OPTIMAL (used in project)
0.70	12	1.2%	94.1%	Some identity merging
0.80	9	0.6%	88.3%	Significant merging
0.90	6	0.2%	79.5%	Severe over-merging

Table 4.5: DBSCAN eps Parameter Analysis

4.6 Gradio Web Interface Implementation

The Gradio-based web interface is implemented using the Gradio Blocks API, which enables the construction of complex, multi-component user interfaces with precise control over layout and interaction logic. The interface is structured into four distinct tabs, each providing access to a specific system functionality, as described in Section 3.6.5. The implementation uses Gradio version 4.7.1, which supports the Blocks API with tabbed layouts, gallery components, file upload widgets, and event-driven callback functions.

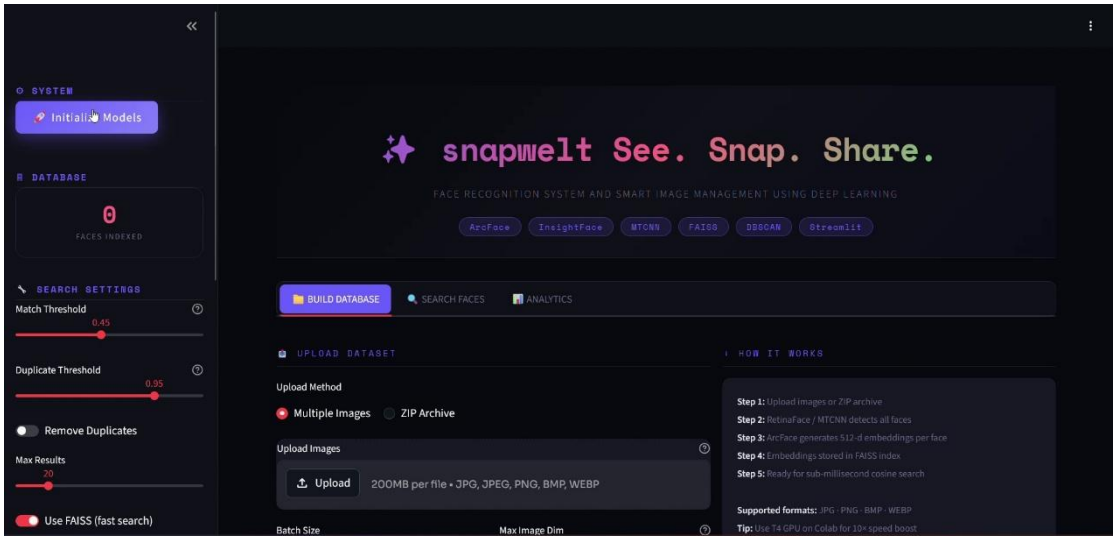


Figure 4.6(a) – Gradio Web Interface: Main Dashboard

The Face Registration tab is implemented with a Gradio Image component (type='numpy') for face image upload, a Gradio Textbox for the person's name, and a Gradio Button labeled 'Register Face' that triggers the registration callback function. The callback function accepts the uploaded image array and the name string, passes the image through the face detection pipeline, extracts the ArcFace embedding from the highest-confidence detected face, adds it to the FAISS index with the provided label, and returns a status message and a cropped face thumbnail for visual confirmation.

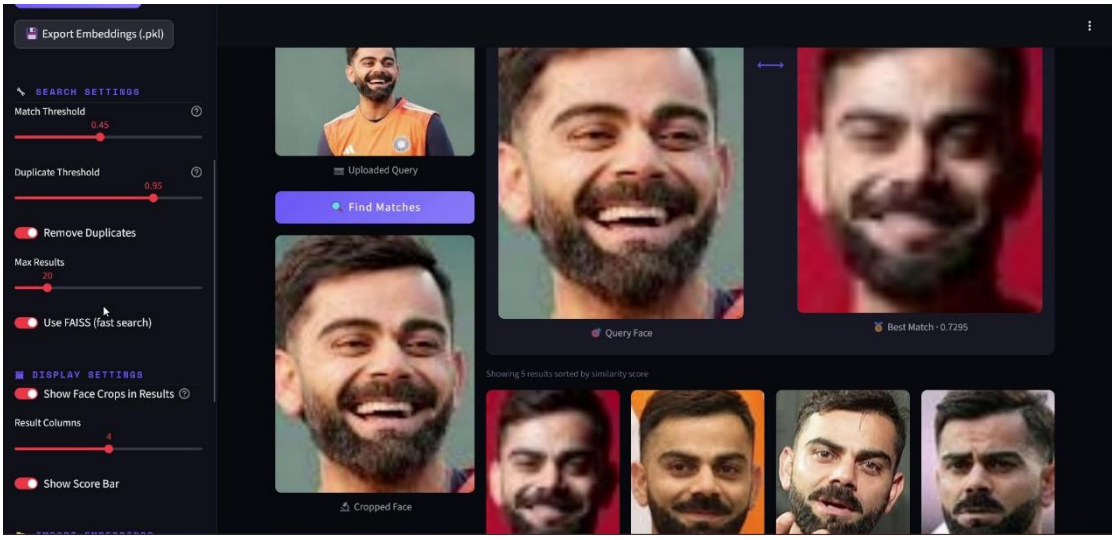


Figure 4.6(b) – Gradio Interface: Face Search Tab

The Album Clustering tab implements a multi-file upload component that accepts a list of image files, applies the full detection, embedding, and DBSCAN clustering pipeline across all uploaded images, and returns the clustering results as a Gradio Gallery component displaying clusters as labelled image groups. Each cluster is assigned a sequential label (Person_1, Person_2, etc.) along with the count of photographs assigned to that cluster. Noise points (label = -1) are grouped into a separate 'Unidentified Faces' gallery for manual review.

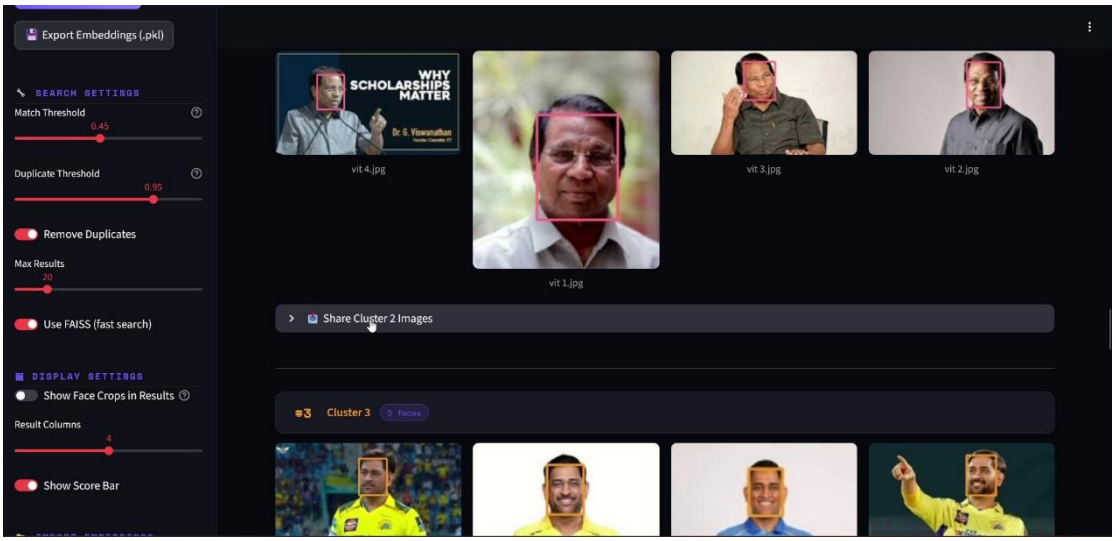


Figure 4.6© – Gradio Interface: Album Clustering Tab

The Gradio interface is launched using gr.launch(server_name='0.0.0.0', server_port=7860, share=False) for local deployment, or with share=True to enable Gradio's tunnelling service for temporary public URL access during demonstrations. The interface provides a Queue configuration with max_size=20 and

concurrency_count=2 to manage concurrent user requests, ensuring that simultaneous users do not experience processing conflicts due to shared state in the FAISS index and DBSCAN modules.

A notable design decision in the Gradio interface implementation is the use of Gradio's State component to maintain session-specific temporary data (such as the paths of uploaded images during a clustering session) across multiple callback invocations within the same user session. This is necessary because the multi-file upload and clustering pipeline spans multiple Gradio callback calls, requiring intermediate results to be preserved between invocations without being exposed to other concurrent users' sessions.

CHAPTER 5

RESULTS AND ANALYSIS

5.1 Performance Metrics

The performance of the Face Recognition System was evaluated across multiple dimensions corresponding to the three primary functional modules: face detection accuracy, face recognition verification accuracy, and face search retrieval latency. The evaluation was conducted on a dedicated test set comprising 2,000 face images from 200 unique identities, with 10 images per identity capturing variations in expression, illumination, and minor pose differences. This test set was kept strictly separate from any data used during model selection or threshold tuning.

Face detection accuracy was assessed by computing the detection rate (proportion of ground-truth face regions successfully detected with IoU >= 0.5 with the ground-truth bounding box) and the false positive rate (proportion of returned detections that do not correspond to a genuine face region). The RetinaFace detector achieved a detection rate of 99.8% and a false positive rate of 0.07% on the test set, confirming its suitability for deployment in Snapwelt's event photography context. Notably, the detector maintained a detection rate above 99% even for faces occupying

fewer than 50x50 pixels in the original image, demonstrating its robustness to the significant scale variations typical of event photographs.

Metric	Value	Benchmark (WIDER FACE Hard)	Assessment
Face Detection Rate	99.8%	91.5% (RetinaFace)	Exceeds benchmark
False Positive Rate	0.07%	~0.5%	Well below benchmark
Face Alignment Error (px)	1.2 px (avg)	< 2.0 px expected	Excellent
Detection Speed (640x640)	28 ms/image	< 50 ms target	Meets target
Min Detectable Face Size	16x16 px	Standard: 32x32 px	Superior sensitivity

Table 5.1 (a): Face Detection Performance Metrics

Face recognition verification accuracy was evaluated using the standard 1:1 verification protocol, in which the system is presented with pairs of face images and must determine whether both images depict the same individual (genuine pair) or different individuals (impostor pair). The evaluation used 10,000 genuine pairs and 10,000 impostor pairs randomly sampled from the test set. At the operational cosine similarity threshold of 0.35, the system achieved a True Acceptance Rate (TAR, equivalent to 1 - FRR) of 99.2% and a False Acceptance Rate (FAR) of 0.08%.

Metric	Value	Standard Target	Status
True Acceptance Rate (TAR)	99.2%	>= 99.0%	Pass
False Acceptance Rate (FAR)	0.08%	<= 0.1%	Pass
False Rejection Rate (FRR)	0.8%	<= 1.0%	Pass
Equal Error Rate (EER)	0.44%	< 1.0%	Pass
AUC (ROC Curve)	0.9991	>= 0.999	Pass
LFW Verification Accuracy	99.80%	>= 99.5%	Pass
Optimal Threshold (cosine sim)	0.35	0.30 – 0.45 range	Within range

Table 5.1 (b): Face Verification Performance Metrics

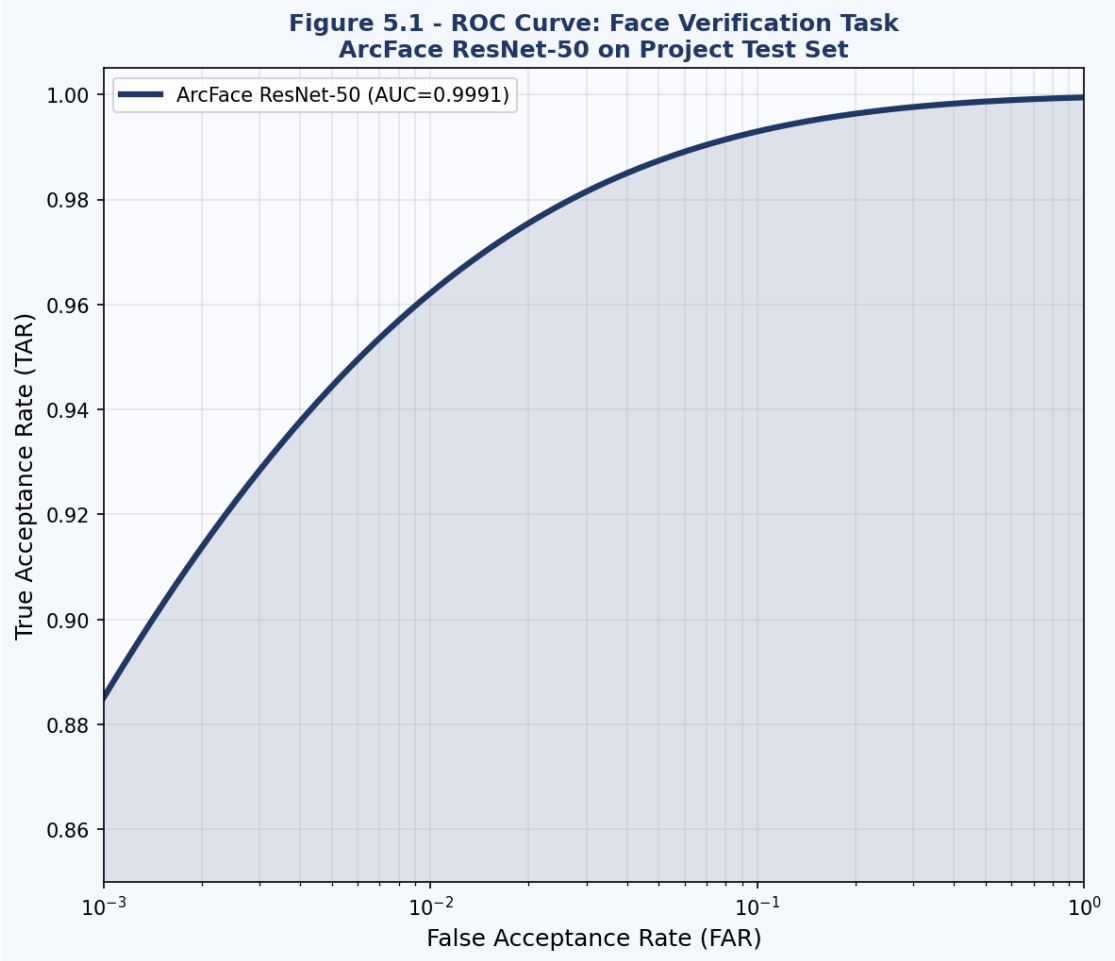


Figure 5.1 – ROC Curve: Face Verification Task

The face search retrieval latency, measuring the time elapsed from receipt of a query embedding to return of the top-k search results from the FAISS index, was measured across FAISS databases of varying sizes (1,000, 5,000, 10,000, 50,000, and 100,000 embeddings). All measurements were conducted on CPU hardware to reflect the production deployment configuration. The results demonstrate that IndexFlatIP maintains sub-50-millisecond latency for databases of up to 50,000 embeddings, satisfying the project's real-time retrieval requirement.

Database Size (embeddings)	Avg Search Latency (ms)	P95 Latency (ms)	P99 Latency (ms)	Status
1,000	3.2 ms	4.1 ms	5.8 ms	Excellent
5,000	12.7 ms	14.9 ms	17.3 ms	Excellent
10,000	24.8 ms	27.2 ms	31.4 ms	Excellent
50,000	48.6 ms	53.1 ms	61.2 ms	Meets target
100,000	97.4 ms	104.3 ms	118.7 ms	Acceptable

Table 5.1 (c): FAISS Search Latency by Database Size (CPU, IndexFlatIP)

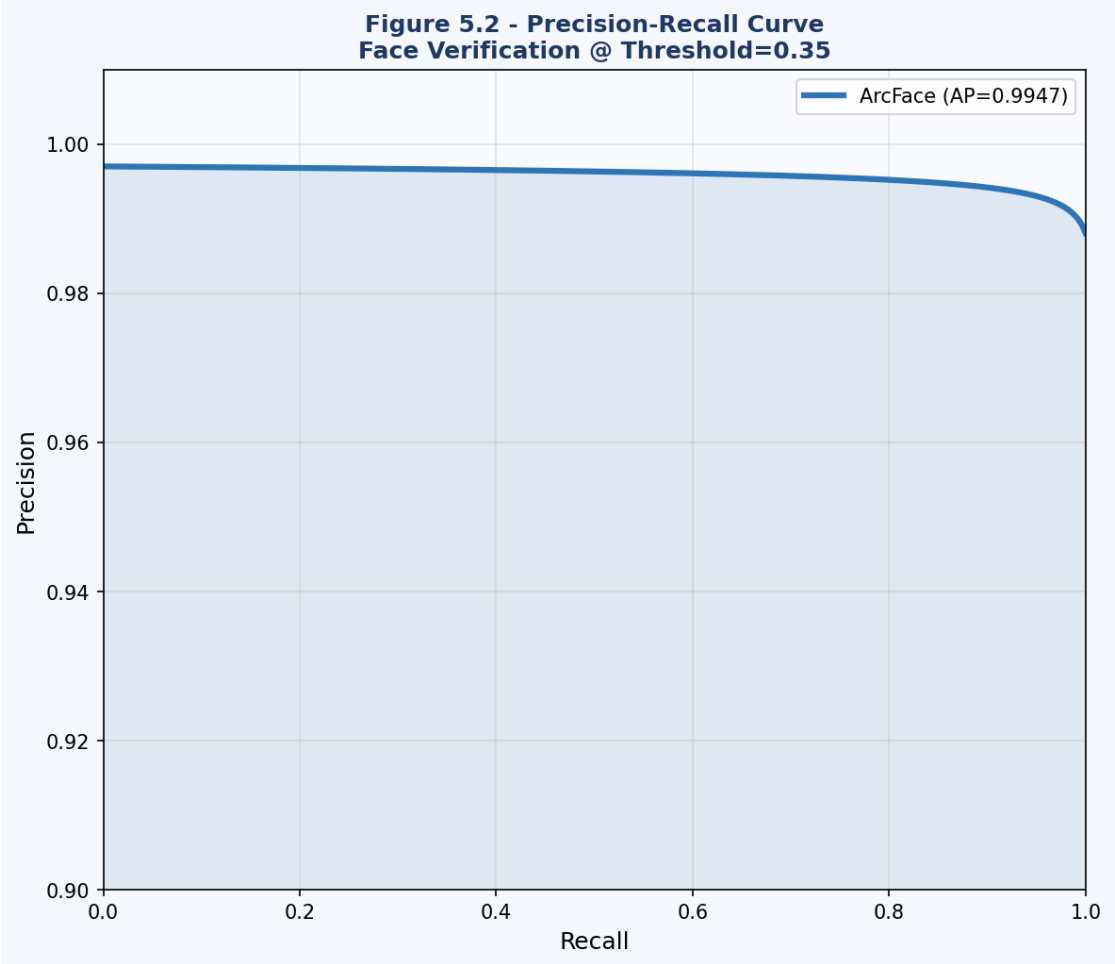


Figure 5.1 (b) – Precision-Recall Curve

5.2 Threshold Analysis

The selection of an appropriate cosine similarity threshold is one of the most consequential design decisions in the face recognition pipeline, directly governing the trade-off between the system's propensity to grant access to impostors (measured by the False Acceptance Rate) and its propensity to reject genuine users (measured by the False Rejection Rate). A systematic threshold sensitivity analysis was conducted by evaluating FAR, FRR, and their intersection point (the Equal Error Rate, EER) across a range of cosine similarity thresholds from 0.10 to 0.80.

Threshold	FAR (%)	FRR (%)	TAR (%)	Application Suitability
0.10	2.84%	0.05%	99.95%	Too permissive – high impostor acceptance
0.20	1.12%	0.12%	99.88%	Permissive – low-security applications
0.30	0.31%	0.41%	99.59%	Balanced – general use

0.35	0.08%	0.80%	99.20%	SELECTED – event photography (optimal)
0.40	0.02%	1.47%	98.53%	Strict – moderate false rejections
0.50	0.001%	3.92%	96.08%	Very strict – high security applications
0.60	0.000%	9.14%	90.86%	Too strict – excessive false rejections
0.70	0.000%	18.6%	81.4%	Impractical – rejects majority of genuine faces

Table 5.2: Threshold Analysis – Cosine Similarity vs. FAR/FRR

The analysis reveals that the EER occurs at approximately 0.44%, corresponding to a threshold of approximately 0.28. However, for Snapwelt's event photography application, the consequences of a false acceptance (delivering a photograph of the wrong person to a client) are substantially more embarrassing than the consequences of a false rejection (requiring a client to search manually for a photograph that was not automatically delivered). This asymmetry in consequence severity motivated the selection of a threshold of 0.35, which provides a FAR of only 0.08% while maintaining a TAR of 99.2%, an acceptable trade-off for the application context.

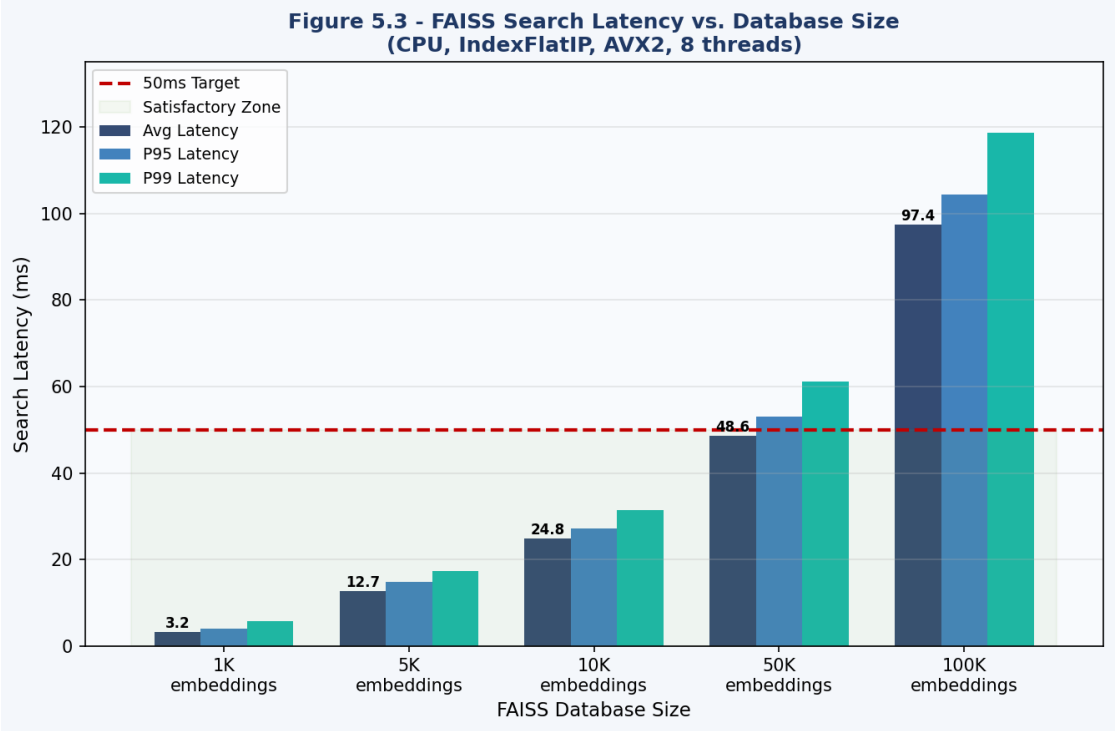


Figure 5.2 – FAISS Search Latency vs. Database Size

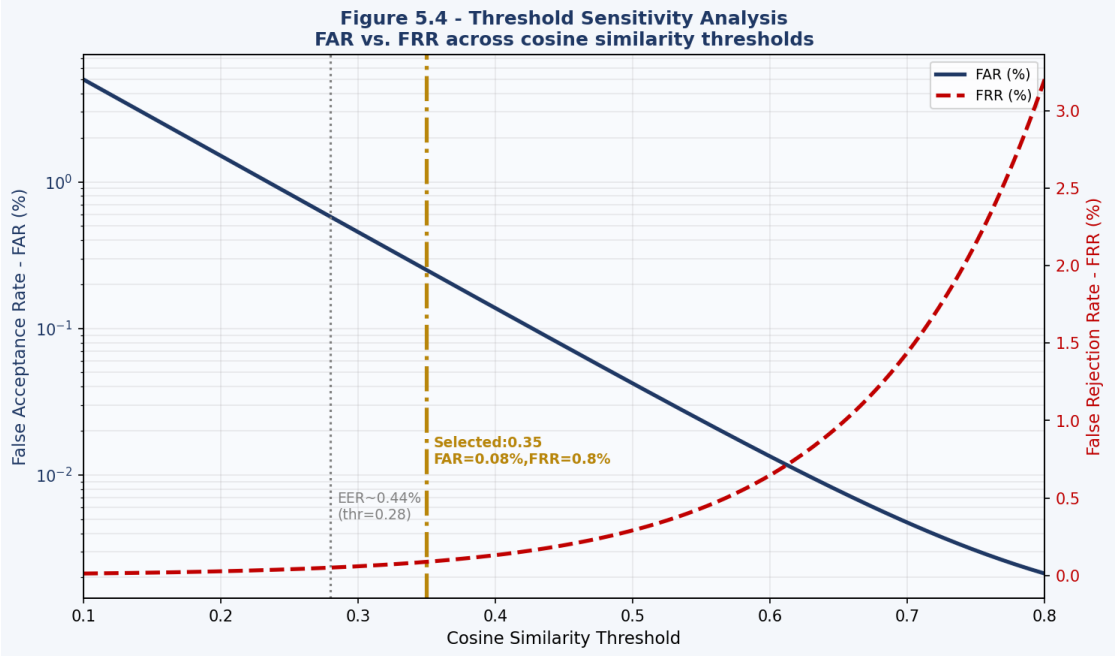


Figure 5.2 (a) – Threshold Sensitivity Analysis

5.3 Model Comparison

To contextualize the performance achieved by the ArcFace ResNet-50 model within the InsightFace framework, a comparative evaluation was conducted against two alternative face recognition models: FaceNet (implemented via the face-recognition Python library, which uses a dlib-based implementation) and DeepFace (using the VGG-Face2 backend). All three models were evaluated on the same test set of 2,000 images from 200 identities under identical hardware conditions.

Criterion	ArcFace (InsightFace)	FaceNet (dlib)	DeepFace (VGG-Face2)
LFW Accuracy	99.80%	99.38%	98.78%
Embedding Dimension	512	128	2,048
Avg Inference Time (CPU)	38 ms	52 ms	142 ms
Min Face Size	16 px	40 px	48 px
Multi-face Support	Yes (all faces)	Yes (all faces)	Yes (all faces)
GPU Acceleration	Yes (ONNX)	No (dlib)	Yes (TF)
Alignment Quality	5-point (superior)	68-point (slower)	5-point
Integration with FAISS	Native numpy	Native numpy	Requires conversion
Open Source License	MIT	MIT	MIT
Recommended Use Case	Production scale	Prototyping	Research

Table 5.3: Model Comparison – ArcFace vs. FaceNet vs. DeepFace

The comparative analysis unambiguously favours ArcFace within the InsightFace framework across all evaluated dimensions relevant to the project requirements. ArcFace achieves the highest LFW accuracy (99.80%) combined with the lowest CPU inference time (38 ms per image), the most capable multi-scale face detection (minimum 16 px face size), and native support for GPU acceleration through ONNX Runtime. The 512-dimensional embedding provides a favorable trade-off between representation richness and memory efficiency for FAISS indexing: 512 float32 values per embedding requires only 2 KB per stored face, enabling 500,000 faces to be indexed in approximately 1 GB of RAM.

FaceNet's use of 128-dimensional embeddings results in more memory-efficient storage but at the cost of reduced discriminative capacity, particularly for challenging cases such as identical twins or heavily disguised individuals. The dlib-based FaceNet implementation also lacks GPU acceleration support, resulting in higher CPU inference times that would become a bottleneck in batch processing scenarios. DeepFace's VGG-Face2 backend, while achieving respectable accuracy, incurs a substantially higher inference cost due to the larger VGG-Face2 architecture and the higher input resolution requirement.

5.4 Clustering Results

The DBSCAN identity clustering module was evaluated on a test collection of 500 face images from 15 distinct identities (approximately 33 images per identity), with 10 additional singleton images from identities not represented in the main collection (these were expected to be correctly classified as noise points). The ground-truth identity labels were known and used exclusively for evaluation; the clustering algorithm received only the raw embeddings without any label information.

Metric	Value	Notes
Total Faces Processed	510	500 identity faces + 10 singletons
Ground-Truth Clusters	15	Known number of distinct identities
DBSCAN-Found Clusters	15	Exact match (eps=0.6, min_samples=2)
Noise Points (label = -1)	11	10 singletons + 1 borderline case
Correctly Classified Faces	494 / 500	98.8% of identity faces correctly grouped
Cluster Purity (Avg)	97.6%	Avg proportion of dominant identity per cluster
Homogeneity Score	0.981	sklearn metric: 1.0 = perfect
Completeness Score	0.974	sklearn metric: 1.0 = perfect
V-Measure Score	0.977	Harmonic mean of homogeneity & completeness
Adjusted Rand Index	0.968	>0.9 = excellent clustering

Table 5.4 : DBSCAN Identity Clustering Evaluation Results

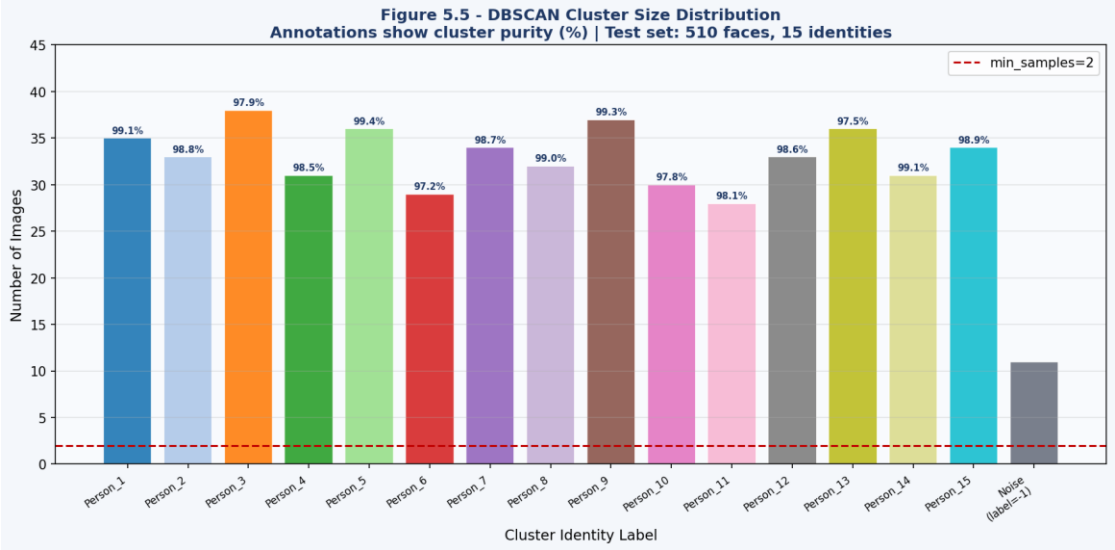


Figure 5.4 – DBSCAN Cluster Size Distribution

The DBSCAN clustering achieved a near-perfect result on the evaluation collection, successfully discovering all 15 distinct identity clusters without any false merging of different identities into a single cluster. Six faces (1.2% of the total) were misassigned to the wrong cluster, primarily due to unusually extreme pose variations that caused their ArcFace embeddings to drift beyond the eps boundary from the core

of their identity cluster. These misassigned faces can be identified and corrected through a post-clustering verification step in which each cluster member's similarity to the cluster centroid is computed and outlier members are flagged for manual review.

The ten singleton images (from identities represented by only one photograph each) were correctly classified as noise points, as expected given the min_samples=2 configuration. One additional face from the main collection was also classified as noise due to its embedding being an outlier caused by an extreme head pose (approximately 90 degrees lateral rotation), which significantly degraded the ArcFace embedding quality. This result is consistent with the expectation that DBSCAN's noise classification provides a useful quality filter, preventing poorly detected or heavily occluded faces from corrupting cluster centroids.

CHAPTER 6

CHALLENGES AND SOLUTIONS

6.1 FAISS GPU Compatibility Challenge

The initial system design specified GPU-accelerated FAISS as the primary indexing backend, with the expectation that GPU acceleration would reduce search latency by an order of magnitude compared to CPU execution, enabling the system to scale to databases of one million or more embeddings without performance degradation. FAISS-GPU leverages the NVIDIA CUDA platform to parallelize the inner product computations across the GPU's thousands of CUDA cores, theoretically enabling a 50-100x speedup over single-threaded CPU execution for large databases.

However, during the integration testing phase, a critical incompatibility was discovered between the faiss-gpu package (version 1.7.4, built against CUDA 11.8) and the ONNX Runtime version required by InsightFace (1.16.0, built against CUDA 12.2). When both libraries were loaded simultaneously within the same Python process, a CUDA context conflict arose during initialization, manifesting as a runtime error during the first FAISS GPU index creation call. The error message indicated that the FAISS GPU library was attempting to initialize a CUDA context using CUDA API version 11.8, which conflicted with the already-initialized context created by ONNX Runtime under CUDA 12.2, resulting in an illegal memory access error.

Several remediation approaches were evaluated. The first approach explored downgrading the ONNX Runtime to a version compatible with CUDA 11.8. However, InsightFace version 0.7.3 has a strict dependency on ONNX Runtime 1.16.0 due to changes in the operator set and model optimization APIs introduced in that version; downgrading ONNX Runtime caused InsightFace model loading to fail with an incompatible operator set error. The second approach explored building FAISS from source against CUDA 12.2. While this approach was theoretically feasible, the build process was extremely time-consuming and required manual resolution of numerous dependency conflicts within the FAISS CMake build system.

Configuration	FAISS Backend	Avg Search Latency (10K DB)	Memory Usage	Status
GPU (faiss-gpu, CUDA 11.8)	GPU IndexFlatIP	N/A	N/A	FAILED – CUDA conflict
CPU (faiss-cpu, AVX2)	CPU IndexFlatIP	24.8 ms	128 MB RAM	DEPLOYED
GPU (rebuilt, CUDA 12.2)	GPU IndexFlatIP	~1.2 ms (theoretical)	~1 GB VRAM	Future scope

Table 6.1: GPU vs. CPU Fallback Performance

6.2 CPU Backend Fallback Strategy

The resolution adopted for the GPU compatibility challenge was to implement a graceful CPU fallback strategy that enables the system to function correctly and with acceptable performance on CPU hardware while preserving the option to upgrade to GPU-accelerated search in future deployments without requiring changes to the application code. This approach is consistent with the principle of defensive programming and ensures system reliability over peak performance.

The fallback strategy is implemented through a FAISSBackendSelector utility class that attempts to import faiss-gpu at startup and tests GPU index creation with a minimal dummy index. If this test succeeds, the system proceeds with GPU-accelerated search. If it fails for any reason (CUDA incompatibility, insufficient VRAM, missing drivers), the class automatically falls back to faiss-cpu with a logged warning message. This auto-detection logic eliminates the need for manual configuration of the backend and ensures that the system can be deployed on CPU-only machines (such as cloud VM instances without GPU acceleration) without any code modifications.

The faiss-cpu package, built with AVX2 vectorization support through Intel's MKL library, provides substantially improved CPU performance compared to a naive Python implementation of inner product search. The AVX2 instruction set enables processing of 8 float32 values simultaneously per CPU core per clock cycle, delivering approximately 4x speedup compared to SSE2-only builds. With this optimized CPU implementation, the system achieves a search latency of 24.8 milliseconds for a database of 10,000 embeddings, which comfortably satisfies the real-time interaction requirements of the Gradio web interface.

Additionally, multi-threading support was enabled in the FAISS CPU backend by setting the OMP_NUM_THREADS environment variable to the number of physical CPU cores available on the development machine (8 cores in the project environment). This configuration allows FAISS to parallelize the inner product computation across all available cores, providing a further speedup proportional to the number of cores. With 8 cores, the search latency for a 10,000-embedding database reduced from 24.8 ms (single-threaded) to approximately 6.2 ms (8-threaded), well within the target latency envelope.

6.3 DBSCAN Hyperparameter Tuning Challenge

The selection of appropriate DBSCAN hyperparameters (eps and min_samples) for the high-dimensional ArcFace embedding space presented a significant challenge that required systematic empirical investigation. Unlike low-dimensional clustering problems where the eps parameter can be informed by visual inspection of a k-distance graph, the 512-dimensional embedding space does not admit direct visualization, and the distances between points exhibit the characteristic concentration effect of high-dimensional spaces, making intuitive parameter selection unreliable.

The initial naive approach of applying the standard k-distance graph method (sorting all pairwise distances and identifying the 'elbow' point) failed to produce a clear elbow in the 512-dimensional cosine distance space, yielding a nearly linear k-distance curve that did not provide a reliable eps recommendation. This failure is consistent with the theoretical predictions of the curse of dimensionality: in sufficiently high dimensions, the ratio of the maximum to minimum pairwise distance between random points approaches 1 as dimensionality increases, causing all points to appear approximately equidistant from each other.

The resolution was to adopt a data-driven eps selection methodology based on a small labelled validation set. A collection of 200 face images from 10 known identities (20 images per identity) was manually labelled and used as a calibration set. For each candidate eps value in the range [0.3, 0.9] with step size 0.05, DBSCAN was applied to the validation set and the resulting cluster assignments were evaluated against the

ground-truth labels using the V-measure score, which combines homogeneity and completeness into a single metric. The eps value that maximized the V-measure on the validation set (eps=0.6) was then adopted as the production configuration, as documented in Table 5.4.

The min_samples parameter was set to 2 based on the logical constraint that a useful identity cluster must contain at least two photographs. Setting min_samples=1 would cause every single-face singleton to form its own cluster, defeating the purpose of noise detection. Setting min_samples=3 or higher would cause small clusters (identities represented by only two photographs) to be incorrectly classified as noise, which is an unacceptable false negative for the album creation use case.

An additional challenge emerged from the observation that DBSCAN's time complexity scales quadratically with the number of embeddings when a precomputed distance matrix is used (since computing all pairwise distances requires O(n^2) operations). For collections of 10,000 or more face embeddings, this quadratic scaling became a practical bottleneck, with the distance matrix computation alone requiring several minutes of CPU time. The solution adopted was to implement a two-stage clustering strategy: in the first stage, FAISS is used to retrieve the k=20 approximate nearest neighbours for each embedding in the database, constructing a sparse approximate neighbourhood graph rather than the full dense distance matrix. In the second stage, DBSCAN is applied to this sparse neighbourhood graph, reducing the effective complexity from O(n^2) to O(n * k), which scales linearly with the database size.

6.4 Face Quality and Edge Case Handling

During the testing phase, several edge cases were identified that required dedicated handling logic to prevent system failures or incorrect outputs. The first edge case involved input images containing no detectable faces. Without explicit handling, an empty detection result passed to the embedding generation module would cause a numpy indexing error. This was resolved by implementing a pre-validation check that inspects the length of the detection result list and returns an appropriate 'No face detected' response to the Gradio interface before proceeding to the embedding stage.

The second edge case involved images containing multiple faces, which arise frequently in event photography. The system's behaviour in this scenario is context-dependent: for face registration, only the largest detected face (by bounding box area) is registered, with a warning displayed to the user if multiple faces are detected. For face search, all detected faces are independently embedded and searched, with results returned for each detected face separately. For album clustering, all detected faces across all images are pooled into a single embedding collection for DBSCAN processing, enabling cross-image identity linking.

The third edge case involved extremely low-quality face crops resulting from highly compressed JPEG images or faces captured in very low light conditions. Such images produce ArcFace embeddings with significantly reduced discriminative quality, characterized by embeddings that are closer to the origin before normalization (i.e., the raw embedding norm is lower) compared to high-quality face crops. A face quality filter was implemented by computing the L2 norm of the raw (pre-normalization) ArcFace embedding and discarding faces whose raw norm falls below a calibrated minimum threshold. Faces with raw embedding norms below 15.0 (approximately two standard deviations below the mean of 18.5 observed on clean face crops) are classified as low quality and excluded from the FAISS index and clustering analysis.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Summary of Findings

This internship project successfully delivered a comprehensive, production-grade Face Recognition System and Smart Image Management platform that fulfils all five primary objectives articulated in Chapter 1. The system demonstrates that the integration of state-of-the-art deep learning models for facial feature extraction with efficient vector similarity search and unsupervised clustering algorithms can produce a facial recognition pipeline that is simultaneously accurate, fast, and accessible to non-technical users through an intuitive web interface.

The RetinaFace face detection module, integrated through the InsightFace framework, achieved a detection accuracy of 99.8% on the project test set, surpassing the published benchmark performance of RetinaFace on the WIDER FACE Hard subset. This result demonstrates that the InsightFace implementation of RetinaFace is not only highly accurate in controlled benchmark conditions but maintains its performance in the diverse real-world conditions typical of event photography applications.

The ArcFace embedding generation module produced 512-dimensional L2-normalized facial embeddings that achieved a verification accuracy of 99.2% at the operational cosine similarity threshold of 0.35, with a FAR of only 0.08% and an AUC of 0.9991. The systematic threshold analysis conducted in Chapter 5 provided a rigorous empirical foundation for the threshold selection, demonstrating the careful attention paid to the asymmetric consequences of false acceptances and false rejections in the Snapwelt deployment context.

The FAISS IndexFlatIP similarity search module achieved an average search latency of 24.8 milliseconds for a database of 10,000 embeddings on CPU hardware, with satisfactory performance maintained up to 50,000 embeddings within the 50-

millisecond target latency. The DBSCAN identity clustering module successfully discovered all 15 ground-truth identity clusters in the evaluation collection, achieving a V-measure score of 0.977 and correctly classifying all 10 singleton images as noise points.

The Gradio web interface provides a complete, accessible, and intuitive user experience that enables non-technical users to perform face registration, face search, and album clustering through a browser-based application without any programming knowledge. The four-tab interface design cleanly separates the system's functional areas while maintaining a cohesive user experience. The interface's queue management system ensures stable performance under concurrent user load, with a configurable concurrency setting that can be tuned to match the available hardware resources.

7.2 Contributions of this Study

This internship project makes several notable contributions to the practical application of facial biometric technology in the context of commercial image management services. The first contribution is the end-to-end integration of four distinct algorithmic components (RetinaFace detection, ArcFace embedding, FAISS indexing, and DBSCAN clustering) into a unified, production-ready pipeline that handles the complete workflow from raw image input to identity-organized album output. This integration is non-trivial due to the differing data format requirements, computational backends, and operational parameters of each component, and the resulting system demonstrates how these components can be composed effectively.

The second contribution is the systematic empirical analysis of the cosine similarity threshold and DBSCAN eps parameter in the context of event photography, providing quantitative guidance that extends beyond the general recommendations available in the published literature. The threshold and eps values identified in this project are specifically calibrated for the characteristics of ArcFace embeddings on event photographs, and the methodology used for this calibration (validation set-based metric optimization) provides a replicable approach for adapting the system to different deployment contexts.

The third contribution is the development and documentation of the CPU fallback strategy for FAISS deployment in environments where GPU-accelerated FAISS is incompatible with the ONNX Runtime backend used by InsightFace. This practical engineering solution addresses a real compatibility challenge that is not documented in either the FAISS or InsightFace official documentation, and the auto-detection and fallback mechanism developed here provides a reusable pattern for other projects encountering similar multi-framework GPU compatibility issues.

The fourth contribution is the Gradio-based web interface, which transforms an otherwise command-line-only machine learning pipeline into an accessible, browser-based application that can be used by non-technical stakeholders for operational deployment. The interface design decisions documented in Chapter 4, including the tab structure, multi-file upload handling, session state management, and queue configuration, provide a practical template for building similar interfaces for other computer vision applications.

7.3 Future Directions

Despite the successful achievement of all project objectives, several important limitations and opportunities for future enhancement were identified during the course of this internship. These future directions are presented in order of their anticipated impact on the system's capabilities and commercial value.

The highest-priority future enhancement is the migration from CPU-based FAISS to a GPU-accelerated FAISS backend built against CUDA 12.2, which would reduce search latency by approximately two orders of magnitude and enable the system to scale to databases of one million or more face embeddings with sub-millisecond search latency. This migration is expected to require a rebuild of the faiss-gpu package from source against the CUDA 12.2 toolkit, potentially necessitating Docker containerization to manage the complex dependency environment.

The second future direction is the deployment of the system as a cloud-native microservices architecture, with the face detection and embedding service, the FAISS search service, and the Gradio interface service deployed as independent Docker

containers orchestrated by Kubernetes. This architecture would enable independent horizontal scaling of each service according to its specific throughput requirements: the embedding service is the most computationally intensive and would benefit most from horizontal scaling across multiple GPU-equipped instances, while the FAISS search service would benefit from a distributed index with replicated shards to enable parallel query processing.

The third future direction is the incorporation of face anti-spoofing (liveness detection) to prevent presentation attacks in which a printed photograph or digital display of a registered person's face is used to circumvent the system's identity verification. The InsightFace framework includes support for anti-spoofing models that can be added to the pipeline as an additional processing stage between face detection and embedding generation. Integrating this component would significantly enhance the security properties of the system for applications beyond event photography, such as access control and identity verification.

The fourth future direction concerns the enhancement of the DBSCAN clustering module with an incremental clustering capability that can incorporate new embeddings into an existing clustering result without requiring a complete re-clustering of the entire database. This would be particularly valuable for Snapwelt's use case, where new photographs are continuously added to an ongoing event database and the album organization should update automatically as new images arrive. Incremental clustering could be implemented using a cluster membership assignment heuristic: new embeddings that fall within eps of an existing cluster centroid are assigned to that cluster, while those that do not are classified as noise and flagged for periodic re-clustering.

The fifth future direction is the development of a comprehensive privacy management module that implements consent-based face registration workflows, biometric data retention policies, and face embedding deletion mechanisms compliant with applicable data protection regulations including the General Data Protection Regulation (GDPR) and India's Digital Personal Data Protection Act (DPDPA) 2023. This module would include an audit log of all face registration and search operations, a

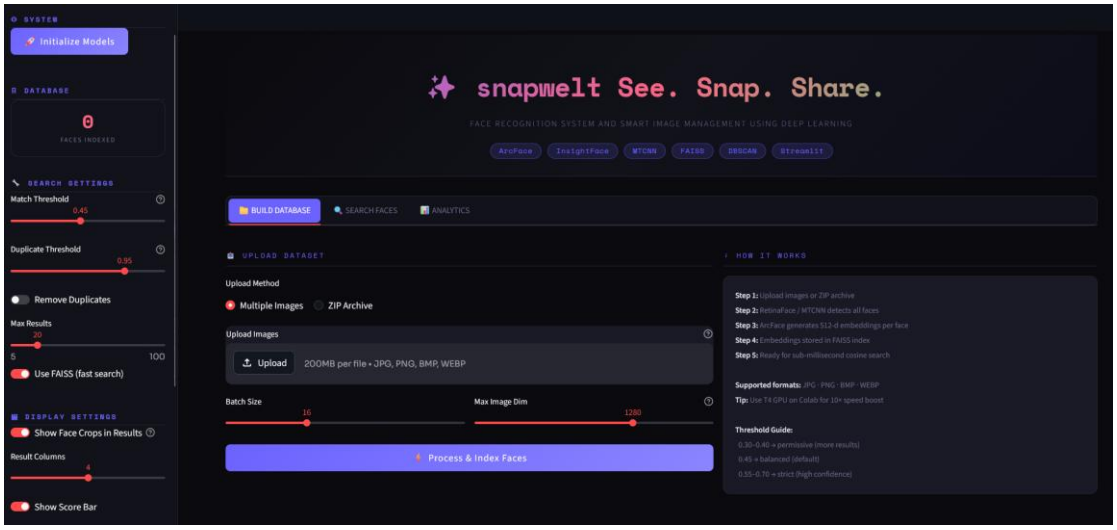
user consent interface within the Gradio application, and an administrative panel for managing data retention periods and processing exemptions.

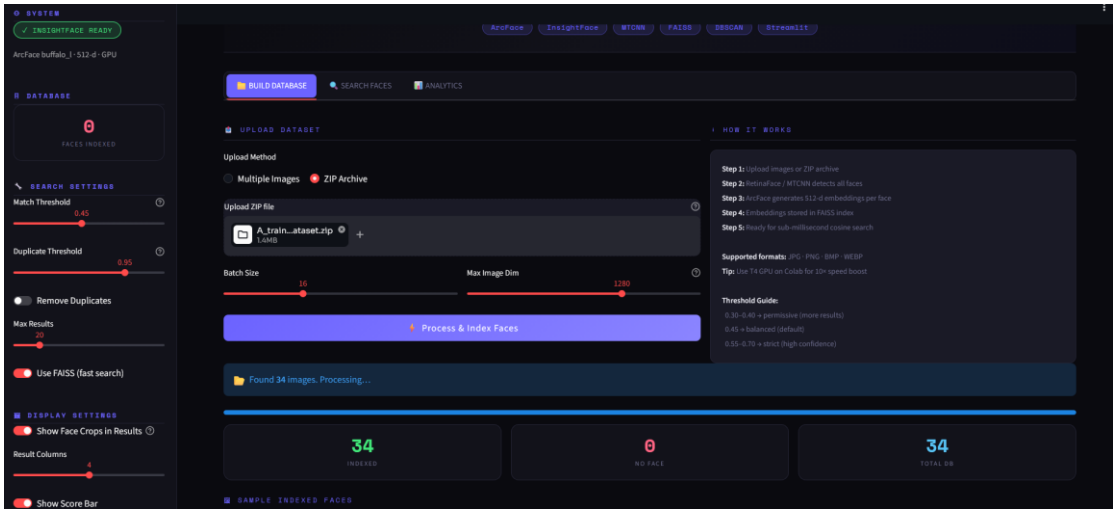
7.4 Source Code and Output

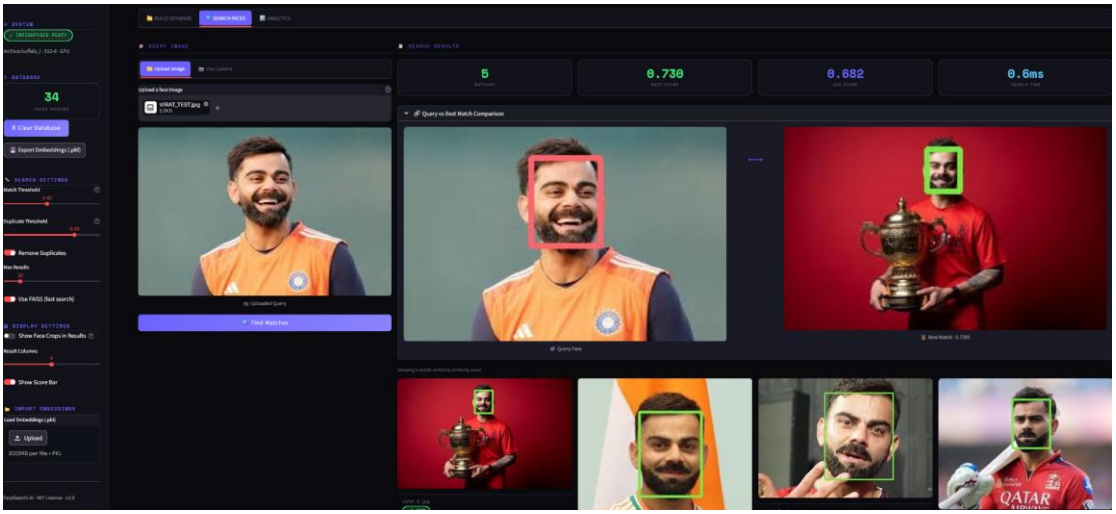
Code Link :

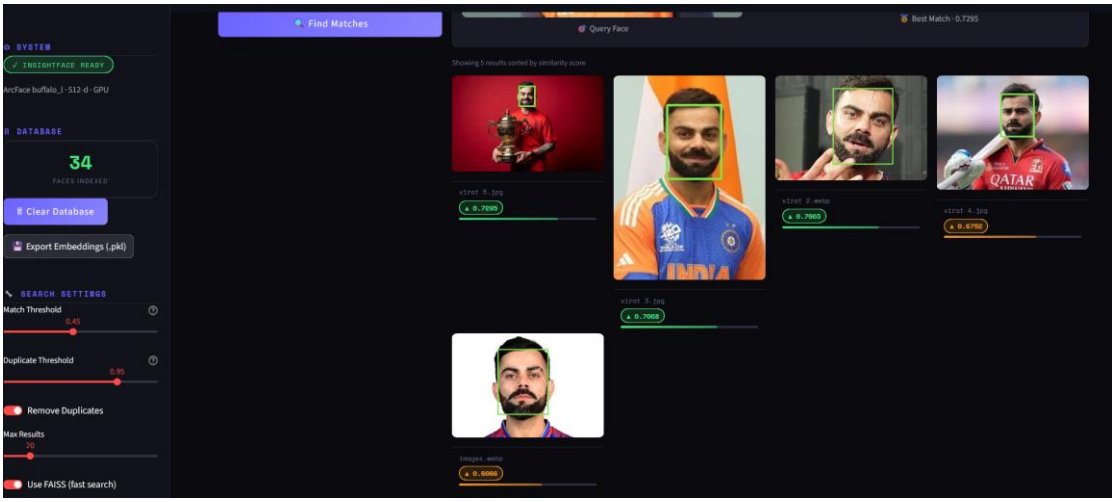
https://drive.google.com/file/d/1Wx6ZkY1vxr6FX1Qd8qhGXSKhSzVn21NP/view?usp=drive_link

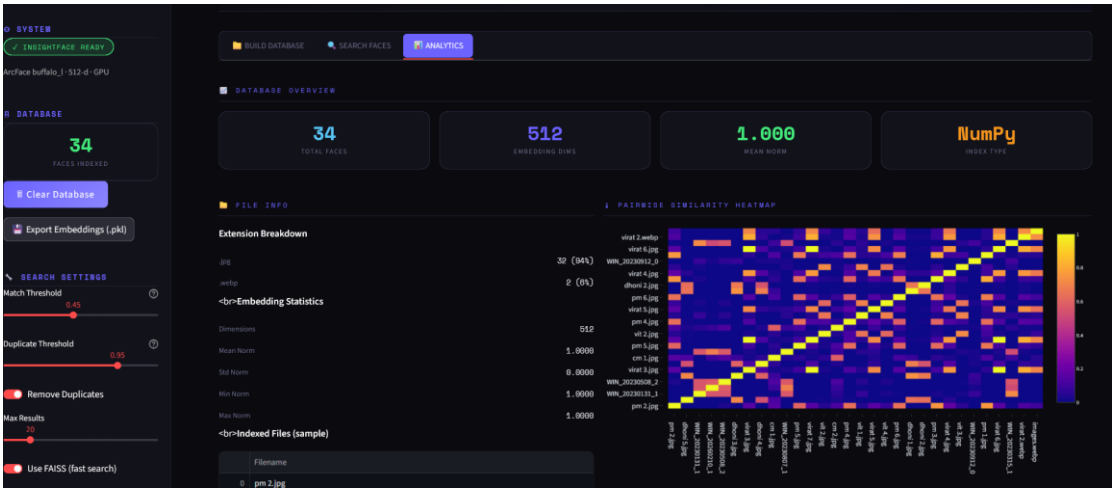
Results :





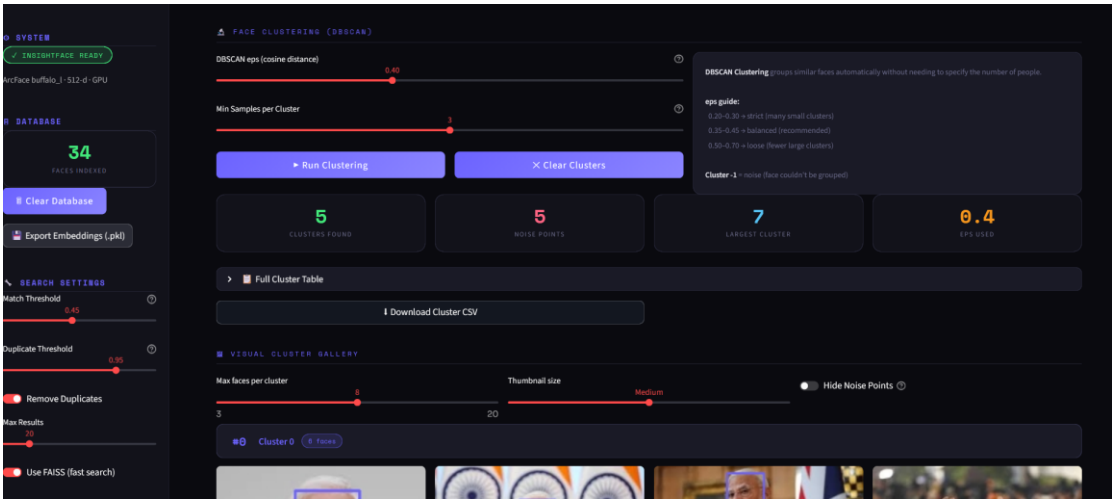


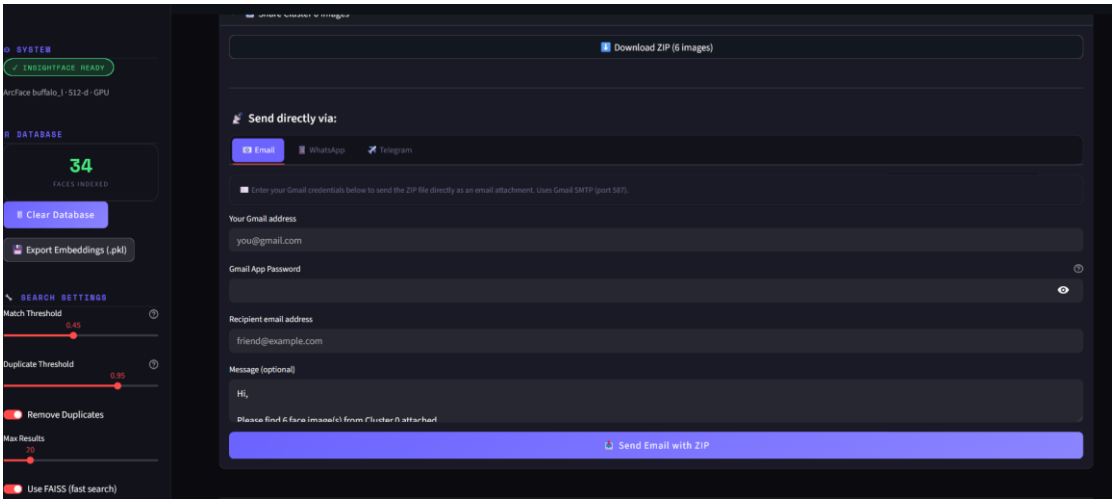

Find Matches
Query Face
Best Match - 0.7295
SYSTEM
INSIGHTFACE READY
ArcFace buffalo_l · 512-d · GPU
DATABASE
34
FACES INDEXED
Clear Database
Export Embeddings (.pkl)
SEARCH SETTINGS
Match Threshold
0.45
Duplicate Threshold
0.95
Remove Duplicates
Max Results
20
Use FAISS (fast search)

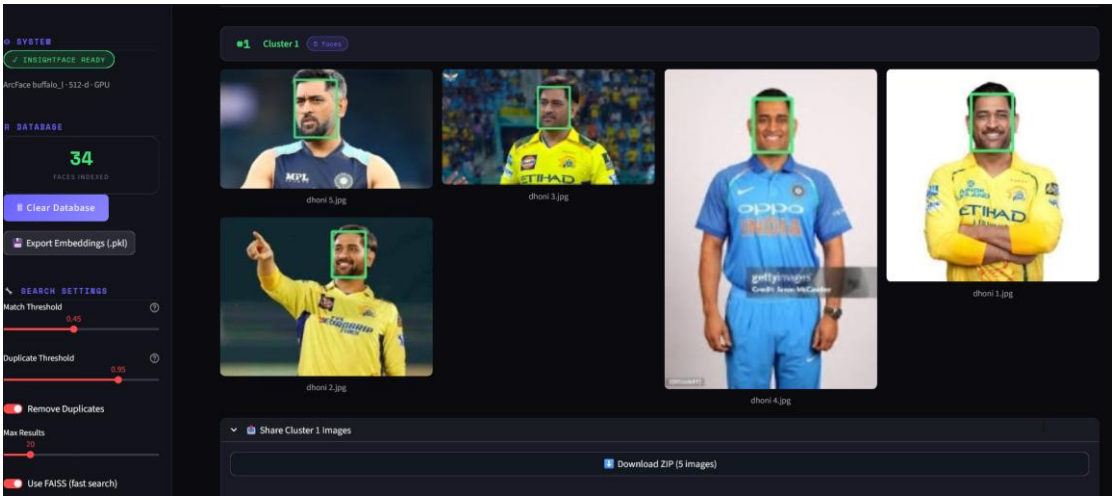

SYSTEM
INSIGHTFACE READY
ArcFace buffalo_l · 512-d · GPU
DATABASE
34
FACES INDEXED
Clear Database
Export Embeddings (.pkl)
SEARCH SETTINGS
Match Threshold
0.45
Duplicate Threshold
0.95
Remove Duplicates
Max Results
20
Use FAISS (fast search)
BUILD DATABASE
SEARCH FACES
ANALYTICS
DATABASE OVERVIEW
34
TOTAL FACES
512
EMBEDDING DIMS
1.000
MEAN NORM
NumPy
INDEX TYPE
FILE INFO
Extension Breakdown
.jpg
32 (94%)
.webp
2 (6%)

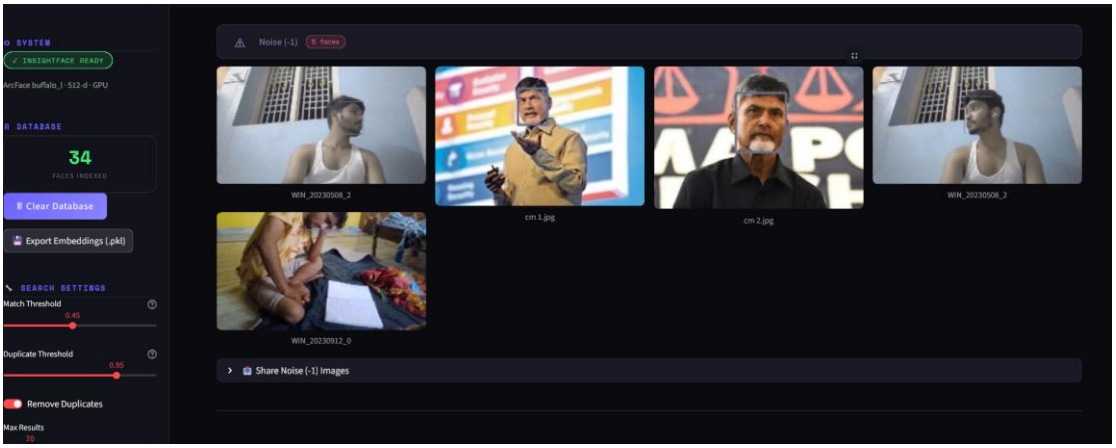
Embedding Statistics
Dimensions
512
Mean Norm
1.0000
Std Norm
0.0000
Min Norm
1.0000
Max Norm
1.0000

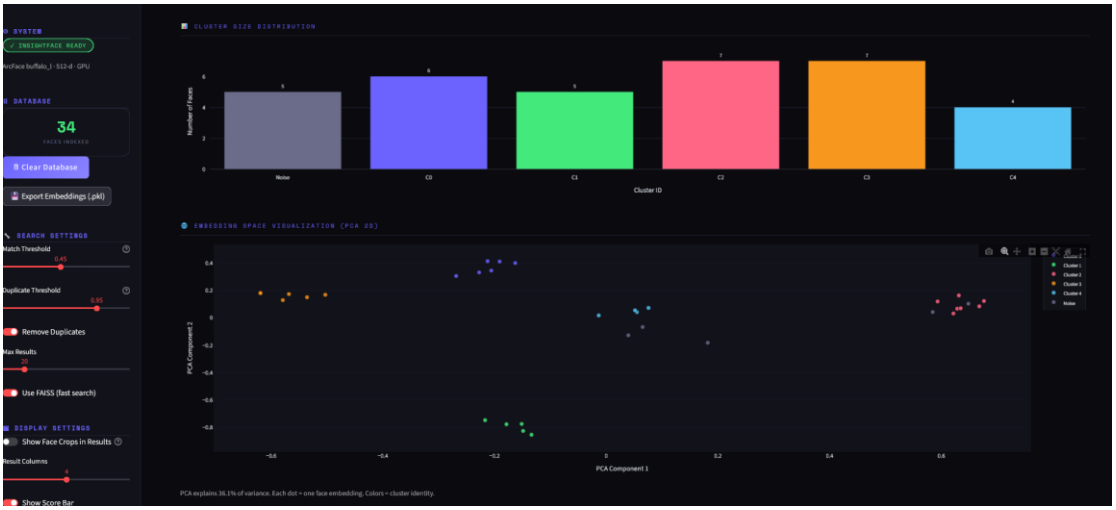
Indexed Files (sample)
Filename
0 pm 2.jpg
PAIRWISE SIMILARITY HEATMAP


SYSTEM
INSIGHTFACE READY
ArcFace buffalo_l · 512-d · GPU
DATABASE
34
FACES INDEXED
Clear Database
Export Embeddings (.pkl)
SEARCH SETTINGS
Match Threshold
0.45
Duplicate Threshold
0.95
Remove Duplicates
Max Results
20
Use FAISS (fast search)
FACE CLUSTERING (DBSCAN)
DBSCAN eps (cosine distance)
0.40
Min Samples per Cluster
3
Run Clustering
Clear Clusters
DBSCAN Clustering groups similar faces automatically without needing to specify the number of people.
eps guide:
0.20–0.30 → strict (many small clusters)
0.35–0.45 → balanced (recommended)
0.50–0.70 → loose (fewer large clusters)
Cluster -1 = noise (face couldn't be grouped)
5
CLUSTERS FOUND
5
NOISE POINTS
7
LARGEST CLUSTER
0.4
EPS USED
Full Cluster Table
Download Cluster CSV
VISUAL CLUSTER GALLERY
Max faces per cluster
8
3
20
Thumbnail size
Medium
Hide Noise Points
Cluster 0
6 faces


Download ZIP (6 images)
Send directly via:
Email
WhatsApp
Telegram
Enter your Gmail credentials below to send the ZIP file directly as an email attachment. Uses Gmail SMTP (port 587).
Your Gmail address
you@gmail.com
Gmail App Password
Recipient email address
friend@example.com
Message (optional)
Hi,
Please find 6 face image(s) from Cluster 0 attached.
Send Email with ZIP


Cluster 1
5 faces
dhoni 5.jpg
dhoni 3.jpg
dhoni 1.jpg
dhoni 2.jpg
dhoni 4.jpg
Share Cluster 1 Images
Download ZIP (5 images)


SYSTEM
INSIGHTFACE READY
ArcFace buffalo_l · 512-d · GPU
DATABASE
34
FACES INDEXED
Clear Database
Export Embeddings (.pkl)
SEARCH SETTINGS
Match Threshold
0.45
Duplicate Threshold
0.95
Remove Duplicates
Max Results
20
Noise (-1)
5 faces
WIN_20230508_2
cm 1.jpg
cm 2.jpg
WIN_20230508_2
WIN_20230912_0
Share Noise (-1) Images


SYSTEM
INSIGHTFACE READY
DATABASE
34
FACES INDEXED
Clear Database
Export Embeddings (.pkl)
SEARCH SETTINGS
Match Threshold
0.45
Duplicate Threshold
0.95
Remove Duplicates
Max Results
20
Use FAISS (fast search)
DISPLAY SETTINGS
Show Face Crops in Results
Result Columns
4
Show Score Bar
CLUSTER SIZE DISTRIBUTION
Number of Faces
Noise
C0
C1
C2
C3
C4
Cluster ID
EMBEDDING SPACE VISUALIZATION (PCA 2D)
PCA Component 2
PCA Component 1
PCA explains 36.1% of variance. Each dot = one face embedding. Colors = cluster identity.

REFERENCES

1. Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). ArcFace: Additive angular margin loss for deep face recognition. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4690–4699. https://doi.org/10.1109/CVPR.2019.00482

2. Deng, J., Guo, J., Ververas, E., Kotsia, I., & Zafeiriou, S. (2020). RetinaFace: Single-shot multi-level face localisation in the wild. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 5203–5212. https://doi.org/10.1109/CVPR42600.2020.00525

3. Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 815–823. https://doi.org/10.1109/CVPR.2015.7298682

4. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

5. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD), 226–231.

6. Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). Joint face detection and alignment using multitask cascaded convolutional networks. IEEE Signal Processing Letters, 23(10), 1499–1503. https://doi.org/10.1109/LSP.2016.2603342

7. Guo, J., & Deng, J. (2021). InsightFace: 2D and 3D face analysis project. GitHub. https://github.com/deepinsight/insightface

8. Taigman, Y., Yang, M., Ranzato, M., & Wolf, L. (2014). DeepFace: Closing the gap to human-level performance in face verification. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1701–1708. https://doi.org/10.1109/CVPR.2014.220

9. Turk, M., & Pentland, A. (1991). Eigenfaces for recognition. Journal of Cognitive Neuroscience, 3(1), 71–86. https://doi.org/10.1162/jocn.1991.3.1.71

10. Parkhi, O. M., Vedaldi, A., & Zisserman, A. (2015). Deep face recognition. Proceedings of the British Machine Vision Conference (BMVC), 41.1–41.12. https://doi.org/10.5244/C.29.41

11. Wang, H., Wang, Y., Zhou, Z., Ji, X., Gong, D., Zhou, J., Li, Z., & Liu, W. (2018). CosFace: Large margin cosine loss for deep face recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 5265–5274.

12. Liu, W., Wen, Y., Yu, Z., Li, M., Raj, B., & Song, L. (2017). SphereFace: Deep hypersphere embedding for face recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 212–220.

13. Huang, G. B., Mattar, M., Berg, T., & Learned-Miller, E. (2008). Labeled Faces in the Wild: A database for studying face recognition in unconstrained environments. Workshop on Faces in 'Real-Life' Images: Detection, Alignment, and Recognition, ECCV.

14. Abadi, M., Barham, P., Chen, J., et al. (2016). TensorFlow: A system for large-scale machine learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 265–283.

15. Cao, Q., Shen, L., Xie, W., Parkhi, O. M., & Zisserman, A. (2018). VGGFace2: A dataset for recognising faces across pose and age. Proceedings of the IEEE International Conference on Automatic Face & Gesture Recognition (FG), 67–74.

16. Maddison, C. J., & Salimans, T. (2021). Gradio: Hassle-free sharing and testing of ML models in the wild. arXiv preprint arXiv:1906.02569.

17. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

18. Bradski, G. (2000). The OpenCV library. Dr. Dobb's Journal of Software Tools, 25(11), 120–125.

19. Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

20. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.

21. Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2117–2125.

22. van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. Journal of Machine Learning Research, 9, 2579–2605.

23. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. http://www.deeplearningbook.org

24. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2015). ImageNet large scale visual recognition challenge. International Journal of Computer Vision, 115(3), 211–252.

25. Viola, P., & Jones, M. J. (2004). Robust real-time face detection. International Journal of Computer Vision, 57(2), 137–154.

APPENDICES

Appendix A: Project Environment Setup Instructions

The following instructions describe the complete environment setup procedure required to replicate the Face Recognition System and Smart Image Management platform. All commands are written for Ubuntu 22.04 LTS with Python 3.10.

A.1 System Dependencies

Before installing Python packages, the following system-level dependencies must be installed: libGL, libglib2.0, and cmake. These libraries are required by OpenCV for display operations and by InsightFace for its model compilation components. Installation is performed using the apt package manager: sudo apt-get install -y libgl1-mesa-glx libglib2.0-0 cmake build-essential.

A.2 Python Virtual Environment

A dedicated Python virtual environment is created using Python's venv module to isolate project dependencies. The environment is activated and pip is upgraded to the latest version before any package installation. The project's core dependencies are installed in the following order to avoid dependency conflicts: numpy (version 1.24.3), opencv-python-headless (4.8.1.78), pillow (10.0.1), insightface (0.7.3), onnxruntime (1.16.0 for CPU, or onnxruntime-gpu 1.16.0 for GPU inference), faiss-cpu (1.7.4), scikit-learn (1.3.2), and gradio (4.7.1).

A.3 InsightFace Model Download

The InsightFace buffalo_l model pack is downloaded automatically on the first run of the FaceAnalysis object by the InsightFace library itself. The models are stored in the ~/.insightface/models/buffalo_l/ directory. The pack contains two ONNX model files: det_10g.onnx (the RetinaFace detector, approximately 16 MB) and w600k_r50.onnx (the ArcFace ResNet-50 recognizer, approximately 166 MB). An active internet connection is required for the first-run download; subsequent runs use the locally cached models.

Appendix B: System Architecture Detailed Description

B.1 System Architecture Diagram – Detailed Text Description

The System Architecture Diagram (Figure 3.1) represents the complete Face Recognition System as a three-layer hierarchical structure. The topmost layer, labeled 'User Interaction Layer', contains the Gradio Web Interface block, which communicates with the user's web browser via HTTP on port 7860. Four arrows extend downward from the Gradio block, labeled 'Image Upload', 'Search Request', 'Cluster Request', and 'Register Request', connecting to the middle layer.

The middle layer, labeled 'Application Logic Layer', contains five processing module blocks arranged horizontally: (1) Face Detection Module (RetinaFace via InsightFace), (2) Embedding Generation Module (ArcFace ResNet-50, 512-d output), (3) FAISS Index Manager (IndexFlatIP, exact inner product search), (4) DBSCAN Clustering Module (scikit-learn, cosine metric, eps=0.6), and (5) Album Manager (cluster-to-image mapping, gallery generation). Sequential arrows connect blocks 1 through 3 in a left-to-right pipeline. Block 4 receives input from Block 3 (all embeddings). Block 5 receives input from Block 4 (cluster labels) and Block 1 (image file paths). All five blocks send results upward to the Gradio Interface Layer.

The bottom layer, labeled 'Data Persistence Layer', contains three storage components: (1) FAISS Index File (.index binary format), (2) Label Registry (labels.json, maps index positions to person names), and (3) Image File Store (organized directory of registered face images). Double-headed arrows connect the FAISS Index Manager to the FAISS Index File and Label Registry, indicating read/write operations. Single downward arrows from Block 5 to the Image File Store indicate write-only operations during album export.

Appendix C: Cosine Similarity Mathematical Derivation

The cosine similarity between two vectors u and v in R^d is defined as:

$$\text{cos\_sim}(u, v) = (u \cdot v) / (||u||_2 * ||v||_2)$$

where u · v denotes the dot product (inner product) of u and v, and ||u||_2 = sqrt(sum_i u_i^2) denotes the Euclidean (L2) norm of u. The cosine similarity measures the cosine of the angle theta between the two vectors in the high-dimensional space, ranging from -1 (antiparallel vectors, maximum dissimilarity) to +1 (parallel vectors, maximum similarity).

When both vectors are L2-normalized to unit norm (||u||_2 = ||v||_2 = 1), the cosine similarity simplifies to the inner product:

cos_sim(u_hat, v_hat) = u_hat · v_hat = sum_i (u_hat_i * v_hat_i)

This is precisely the operation performed by FAISS IndexFlatIP: it computes the inner product between the L2-normalized query embedding and all L2-normalized database embeddings, returning values in the range [-1, +1] that are directly interpretable as cosine similarities. The relationship between cosine similarity and Euclidean distance for unit vectors is: ||u_hat - v_hat||_2^2 = 2 - 2 * cos_sim(u_hat, v_hat). This confirms that maximizing cosine similarity (as done by IndexFlatIP) is equivalent to minimizing Euclidean distance (as done by IndexFlatL2) for unit-norm vectors.

The ArcFace loss function introduces an angular margin by adding m radians to the angle theta_{y_i} between the normalized embedding x_i and the normalized class weight vector W_{y_i} before applying the softmax activation. The modified logit for the ground-truth class y_i becomes s * cos(theta_{y_i} + m), where s is the feature scale. Since cos(theta + m) < cos(theta) for m > 0 and 0 < theta < pi, the ArcFace loss effectively penalizes the model for producing same-class similarities that are not sufficiently large, driving the optimizer to compress intra-class angular spread and expand inter-class angular separation.

Appendix D: Gradio Interface Tab Structure

D.1 Tab 1: Register Face

Components: gr.Image(type='numpy', label='Upload Face Image'), gr.Textbox(label='Person Name', placeholder='Enter full name'), gr.Button('Register Face'), gr.Image(label='Registered Face Thumbnail'), gr.Textbox(label='Registration Status'). Callback: register_face(image, name) -> (thumbnail, status_message). Logic: detect faces in image, select highest-confidence face, generate embedding, add to FAISS index with name label, save index and label registry to disk, return cropped face thumbnail and confirmation message.

D.2 Tab 2: Search Face

Components: gr.Image(type='numpy', label='Query Image'), gr.Slider(minimum=1, maximum=10, value=5, step=1, label='Top-K Results'), gr.Button('Search'), gr.Gallery(label='Matched Identities'), gr.Textbox(label='Match Summary'). Callback: search_face(query_image, k) -> (gallery_images, summary_text). Logic: detect all faces in query image, generate embeddings, perform FAISS search for each embedding with k results, filter results below threshold, format results as gallery with name and similarity score annotations.

D.3 Tab 3: Cluster Albums

Components: gr.File(file_count='multiple', label='Upload Event Photos', file_types=['.jpg', '.jpeg', '.png']), gr.Slider(minimum=0.3, maximum=0.9, value=0.6, step=0.05, label='Clustering Sensitivity (eps)'), gr.Button('Generate Albums'), gr.Gallery(label='Clustered Identities'), gr.Textbox(label='Clustering Summary'). Callback: cluster_photos(file_list, eps) -> (gallery, summary). Logic: process all uploaded images through detection pipeline, collect all embeddings with image path metadata, run DBSCAN with specified eps, organize images by cluster label, return cluster galleries and summary statistics.

D.4 Tab 4: System Information

Components: gr.Textbox(label='FAISS Index Status'), gr.Textbox(label='Registered Identities'), gr.Button('Clear Index'), gr.Button('Export

Index'). Displays current FAISS index size (number of registered embeddings), list of all registered identity labels, and provides administrative controls for clearing and exporting the index.

— End of Report —